

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]

16

Concurrency Control Theory



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

ADMINISTRIVIA

Project #3 is due Sun Nov 17th @ 11:59pm.

Homework #4 will be released next week.
It is due Wed Nov 13th @ 11:59pm.



COURSE STATUS

A DBMS's concurrency control and recovery components permeate throughout the design of its entire architecture.

Query Planning

Operator Execution

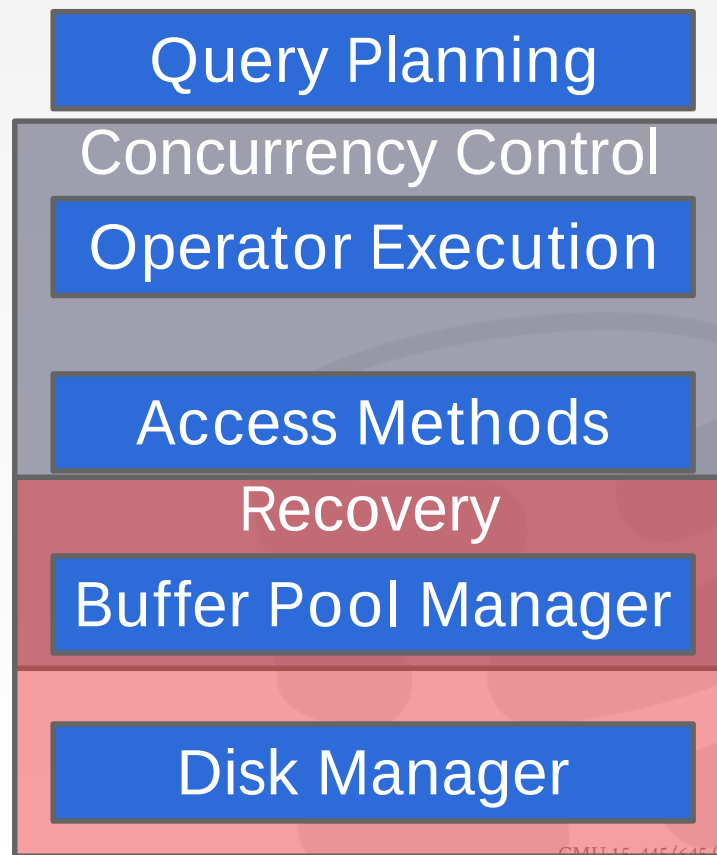
Access Methods

Buffer Pool Manager

Disk Manager

COURSE STATUS

A DBMS's concurrency control and recovery components permeate throughout the design of its entire architecture.



MOTIVATION

We both change the same record in a table at the same time.

How to avoid race condition?



Lost Updates
Concurrency Control

You transfer \$100 between bank accounts but there is a power failure.

What is the correct database state?



Durability
Recovery

CONCURRENCY CONTROL & RECOVERY

Valuable properties of DBMSs.

Based on concept of transactions with **ACID** properties.

Let's talk about transactions...



TRANSACTIONS

A **transaction** is the execution of a sequence of one or more operations (e.g., SQL queries) on a database to perform some higher-level function.

It is the basic unit of change in a DBMS:
→ Partial transactions are not allowed!

TRANSACTION EXAMPLE

Move \$100 from Andy's bank account to his promotor's account.

Transaction:

- Check whether Andy has \$100.
- Deduct \$100 from his account.
- Add \$100 to his promotor account.



STRAWMAN SYSTEM

Execute each txn one-by-one (i.e., serial order) as they arrive at the DBMS.

→ One and only one txn can be running at the same time in the DBMS.

Before a txn starts, copy the entire database to a new file and make all changes to that file.

→ If the txn completes successfully, overwrite the original file with the new one.

→ If the txn fails, just remove the dirty copy.

PROBLEM STATEMENT

A (potentially) better approach is to allow concurrent execution of independent transactions.

Why do we want that?

- Better utilization/throughput
- Increased response times to users.

But we also would like:

- Correctness
- Fairness



TRANSACTIONS

Hard to ensure correctness...

→ What happens if Andy only has \$100 and tries to pay off two promoters at the same time?

Hard to execute quickly...

→ What happens if Andy tries to pay off his gambling debts at the exact same time?

PROBLEM STATEMENT

Arbitrary interleaving of operations can lead to:

- Temporary Inconsistency (ok, unavoidable)
- Permanent Inconsistency (bad!)

We need formal correctness criteria to determine whether an interleaving is valid.

DEFINITIONS

A txn may carry out many operations on the data retrieved from the database

However, the DBMS is only concerned about what data is read/written from/to the database.

→ Changes to the "outside world" are beyond the scope of the DBMS.

FORMAL DEFINITIONS

Database: A fixed set of named data objects (e.g., **A, B, C**, ...).

→ We do not need to define what these objects are now.

Transaction: A sequence of read and write operations (**R(A)**, **W(B)**, ...)

→ DBMS's abstract view of a user program

TRANSACTIONS IN SQL

A new txn starts with the **BEGIN** command.

The txn stops with either **COMMIT** or **ABORT**:

- If commit, the DBMS either saves all the txn's changes or aborts it.
- If abort, all changes are undone so that it's like as if the txn never executed at all.

Abort can be either self-inflicted or caused by the DBMS.

CORRECTNESS CRITERIA: ACID

Atomicity: All actions in the txn happen, or none happen.

Consistency: If each txn is consistent and the DB starts consistent, then it ends up consistent.

Isolation: Execution of one txn is isolated from that of other txns.

Durability: If a txn commits, its effects persist.

CORRECTNESS CRITERIA: ACID

Atomicity: “all or nothing”

Consistency: “it looks correct to me”

Isolation: “as if alone”

Durability: “survive failures”



TODAY'S AGENDA

Atomicity

Consistency

Isolation

Durability



ATOMICITY OF TRANSACTIONS

Two possible outcomes of executing a txn:

- Commit after completing all its actions.
- Abort (or be aborted by the DBMS) after executing some actions.

DBMS guarantees that txns are **atomic**.

- From user's point of view: txn always either executes all its actions, or executes no actions at all.

ATOMICITY OF TRANSACTIONS

Scenario #1:

→ We take \$100 out of Andy's account but then the DBMS aborts the txn before we transfer it.

Scenario #2:

→ We take \$100 out of Andy's account but then there is a power failure before we transfer it.

What should be the correct state of Andy's account after both txns abort?

MECHANISMS FOR ENSURING ATOMICITY

Approach #1: Logging

- DBMS logs all actions so that it can undo the actions of aborted transactions.
- Maintain undo records both in memory and on disk.
- Think of this like the black box in airplanes...

Logging is used by almost every DBMS.

- Audit Trail
- Efficiency Reasons



MECHANISMS FOR ENSURING ATOMICITY

Approach #2: Shadow Paging

- DBMS makes copies of pages and txns make changes to those copies. Only when the txn commits is the page made visible to others.
- Originally from System R.

Few systems do this:

- CouchDB
- LMDB (OpenLDAP)



MECHANISMS FOR ENSURING ATOMICITY

Approach #2: Shadow Paging

- DBMS makes copies of pages and txns make changes to those copies. Only when the txn commits is the page made visible to others.
- Originally from System R.

Few systems do this:

- CouchDB
- LMDB (OpenLDAP)



CONSISTENCY

The "world" represented by the database is logically correct. All questions asked about the data are given logically correct answers.

Database Consistency
Transaction Consistency



DATABASE CONSISTENCY

The database accurately models the real world and follows integrity constraints.

Transactions in the future see the effects of transactions committed in the past inside of the database.

TRANSACTION CONSISTENCY

If the database is consistent before the transaction starts (running alone), it will also be consistent after.

Transaction consistency is the application's responsibility.

→ We won't discuss this further...

ISOLATION OF TRANSACTIONS

Users submit txns, and each txn executes as if it was running by itself.

→ Easier programming model to reason about.

But the DBMS achieves concurrency by interleaving the actions (reads/writes of DB objects) of txns.

We need a way to interleave txns but still make it appear as if they ran one-at-a-time.

MECHANISMS FOR ENSURING ISOLATION

A **concurrency control** protocol is how the DBMS decides the proper interleaving of operations from multiple transactions.

Two categories of protocols:

- **Pessimistic**: Don't let problems arise in the first place.
- **Optimistic**: Assume conflicts are rare, deal with them after they happen.

EXAMPLE

Assume at first **A** and **B** each have \$1000.

T₁ transfers \$100 from **A**'s account to **B**'s

T₂ credits both accounts with 6% interest.

T₁

BEGIN

$A = A - 100$

$B = B + 100$

COMMIT

T₂

BEGIN

$A = A * 1.06$

$B = B * 1.06$

COMMIT

EXAMPLE

Assume at first **A** and **B** each have \$1000.

What are the possible outcomes of running T_1 and T_2 ?

T_1

```
BEGIN
A=A-100
B=B+100
COMMIT
```

T_2

```
BEGIN
A=A*1.06
B=B*1.06
COMMIT
```

EXAMPLE

Assume at first **A** and **B** each have \$1000.

What are the possible outcomes of running T_1 and T_2 ?

Many! But **A+B** should be:

→ $\$2000 * 1.06 = \2120

There is no guarantee that T_1 will execute before T_2 or vice-versa, if both are submitted together. But the net effect must be equivalent to these two transactions running serially in some order.

EXAMPLE

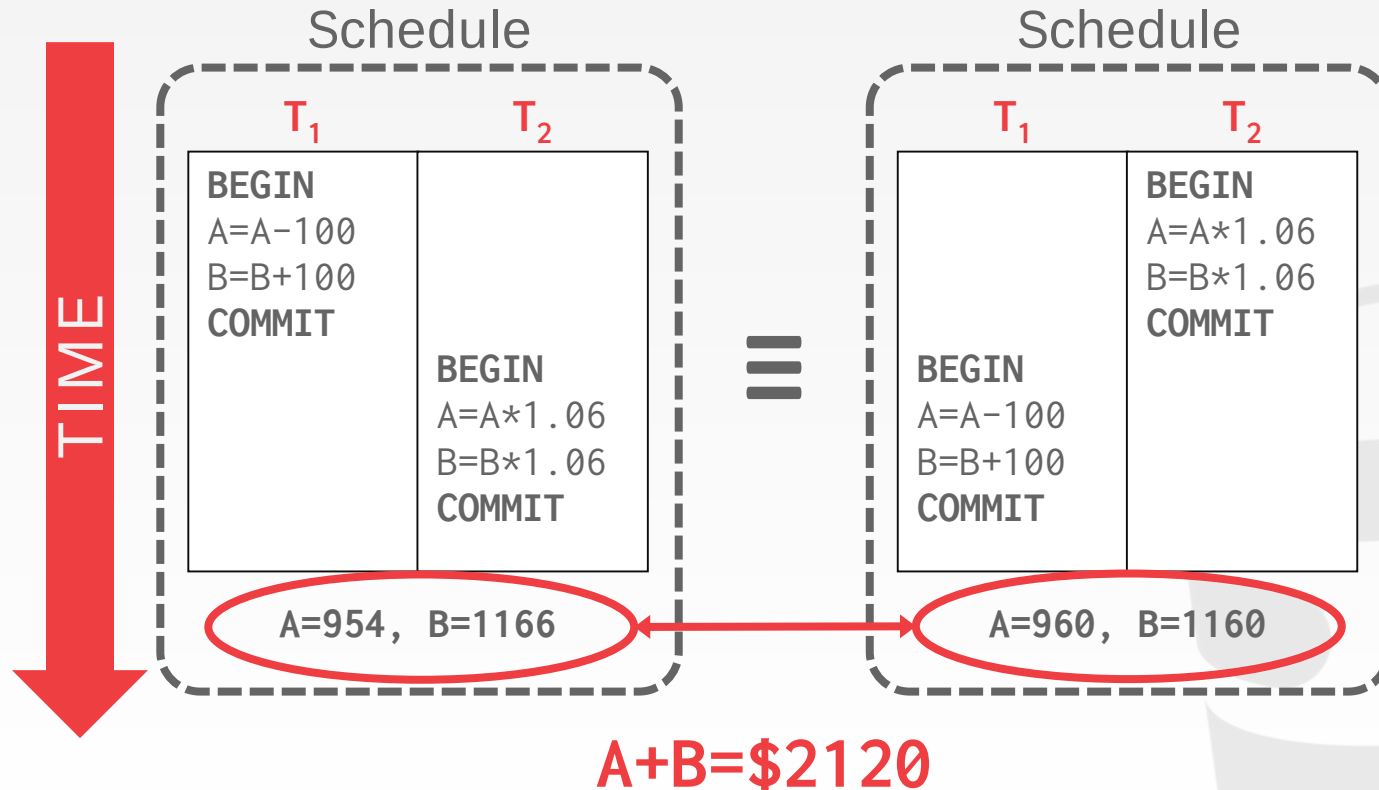
Legal outcomes:

→ $A=954$, $B=1166$ → $A+B=\$2120$

→ $A=960$, $B=1160$ → $A+B=\$2120$

The outcome depends on whether T_1 executes before T_2 or vice versa.

SERIAL EXECUTION EXAMPLE



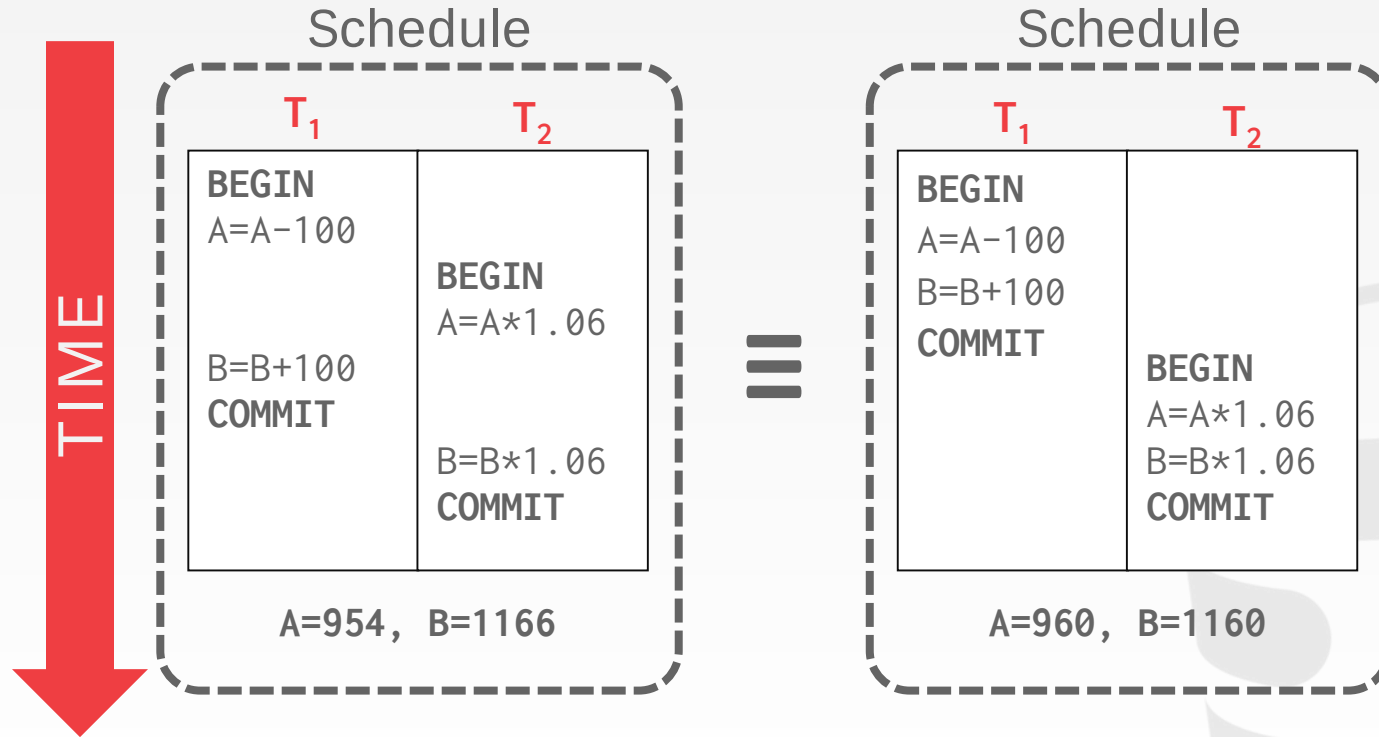
INTERLEAVING TRANSACTIONS

We interleave txns to maximize concurrency.

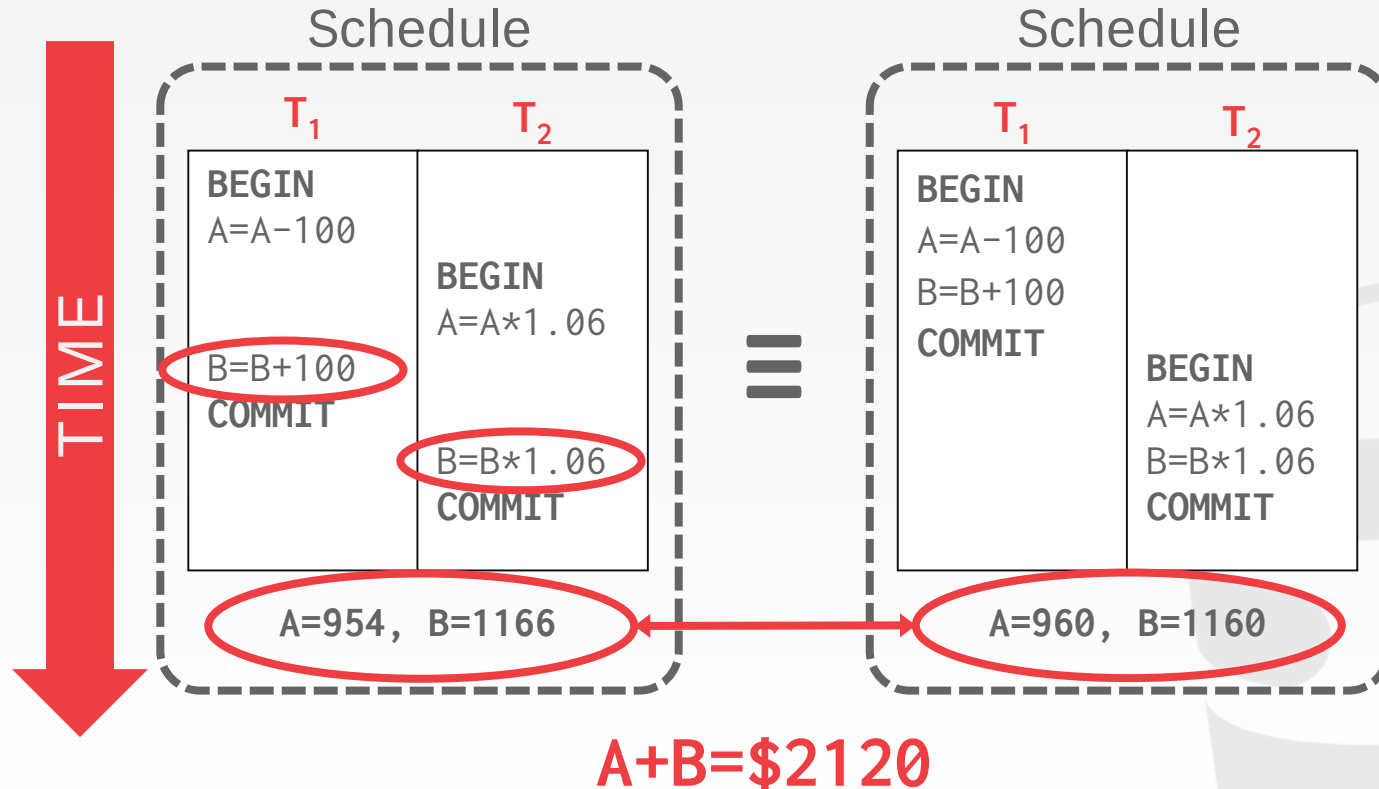
- Slow disk/network I/O.
- Multi-core CPUs.

When one txn stalls because of a resource (e.g., page fault), another txn can continue executing and make forward progress.

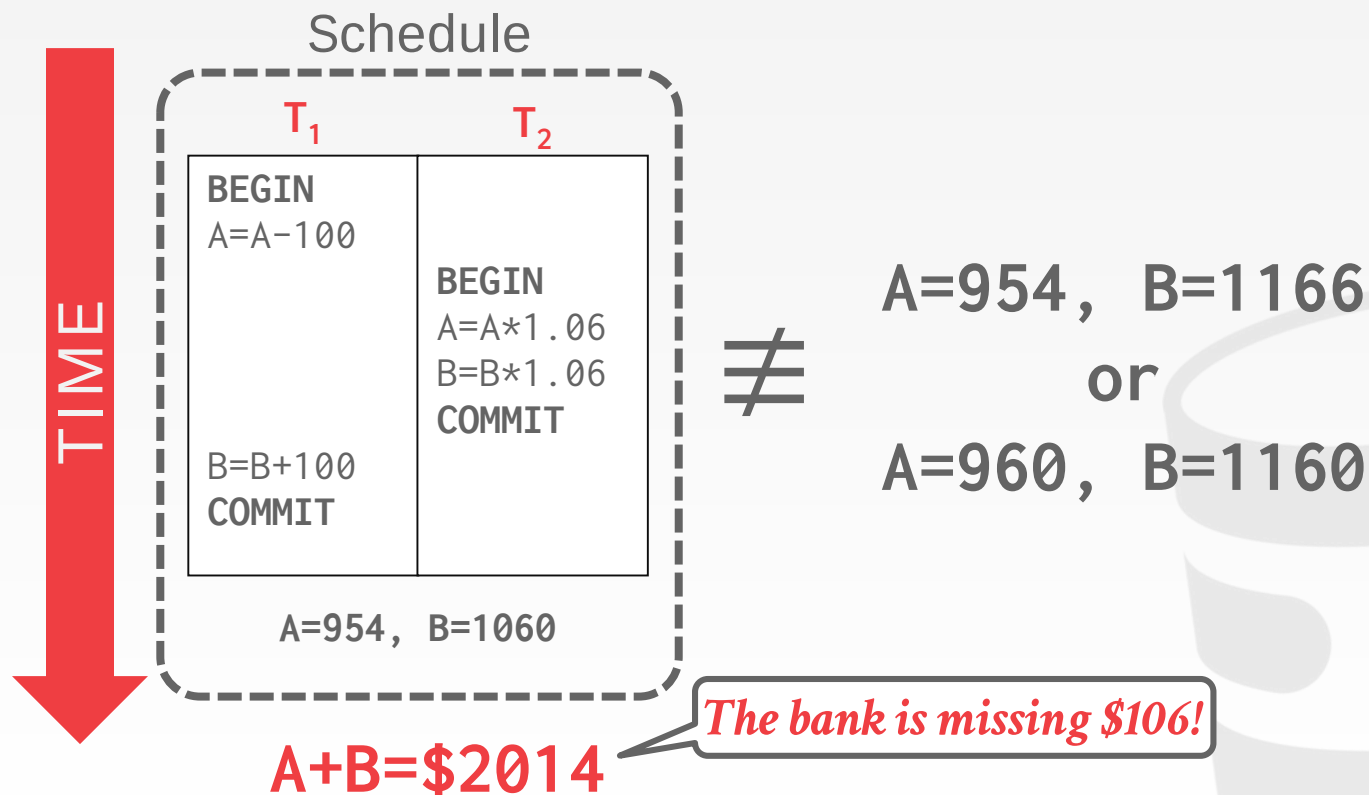
INTERLEAVING EXAMPLE (GOOD)



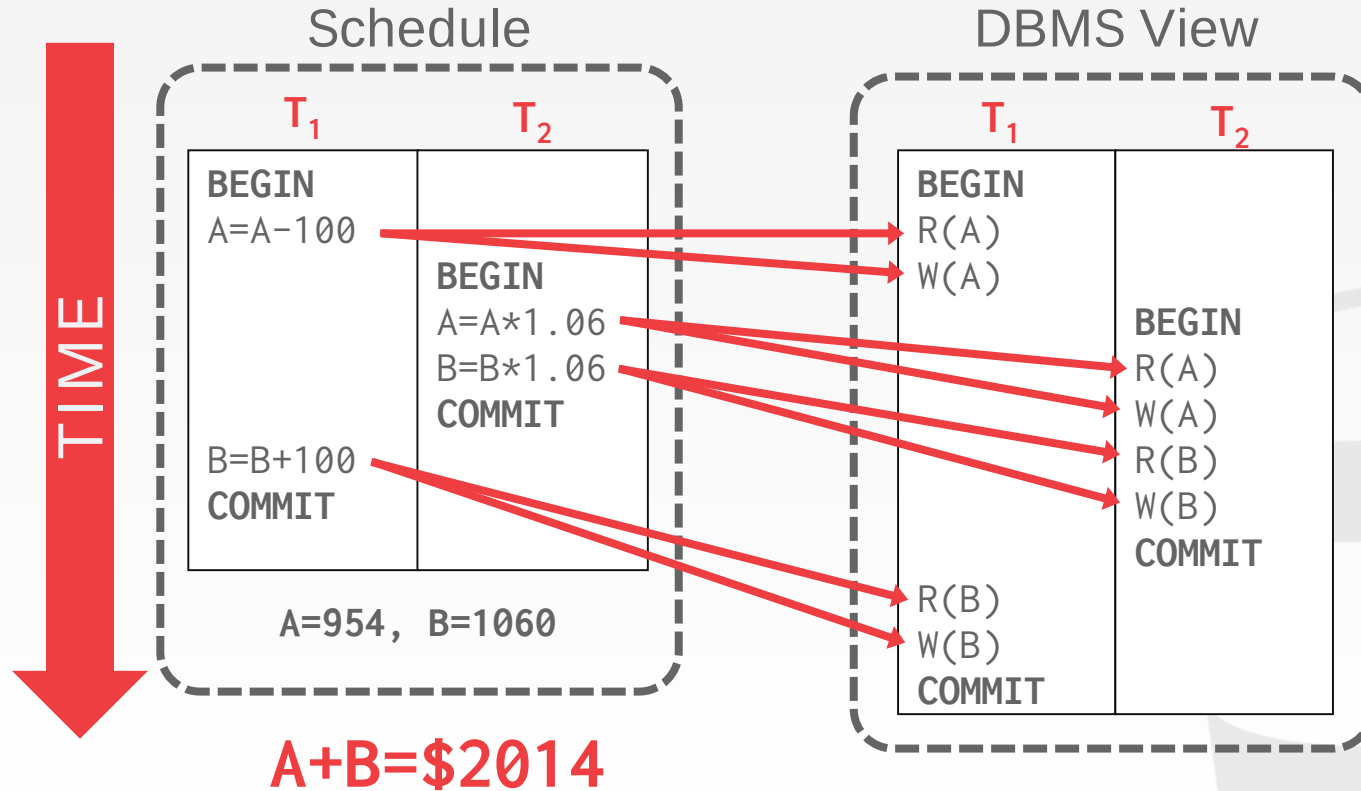
INTERLEAVING EXAMPLE (GOOD)



INTERLEAVING EXAMPLE (BAD)



INTERLEAVING EXAMPLE (BAD)



CORRECTNESS

How do we judge whether a schedule is correct?

If the schedule is **equivalent** to some **serial execution**.

FORMAL PROPERTIES OF SCHEDULES

Serial Schedule

- A schedule that does not interleave the actions of different transactions.

Equivalent Schedules

- For any database state, the effect of executing the first schedule is identical to the effect of executing the second schedule.
- Doesn't matter what the arithmetic operations are!

FORMAL PROPERTIES OF SCHEDULES

Serializable Schedule

→ A schedule that is equivalent to some serial execution of the transactions.

If each transaction preserves consistency, every serializable schedule preserves consistency.

FORMAL PROPERTIES OF SCHEDULES

Serializability is a less intuitive notion of correctness compared to txn initiation time or commit order, but it provides the DBMS with additional flexibility in scheduling operations.

More flexibility means better parallelism.

CONFLICTING OPERATIONS

We need a formal notion of equivalence that can be implemented efficiently based on the notion of "conflicting" operations

Two operations **conflict** if:

- They are by different transactions,
- They are on the same object and at least one of them is a write.

INTERLEAVED EXECUTION ANOMALIES

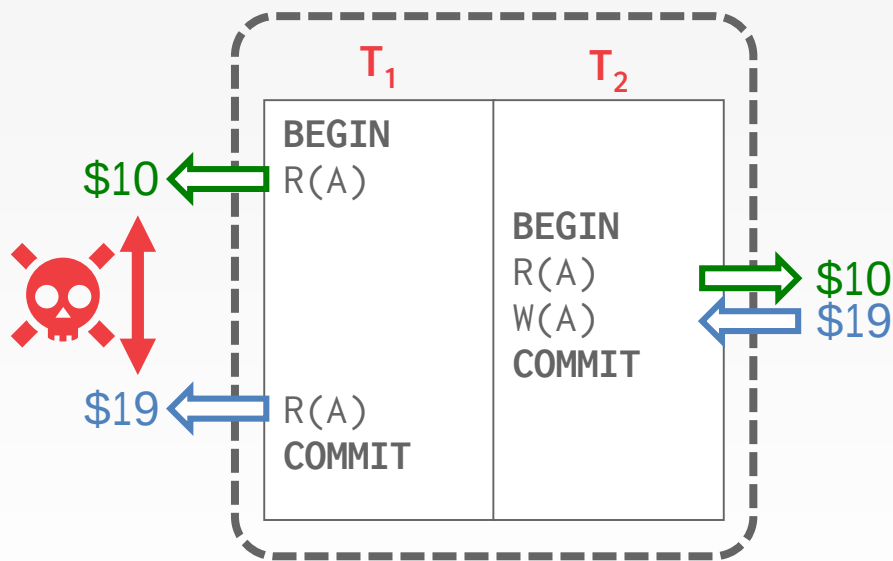
Read-Write Conflicts (**R-W**)

Write-Read Conflicts (**W-R**)

Write-Write Conflicts (**W-W**)

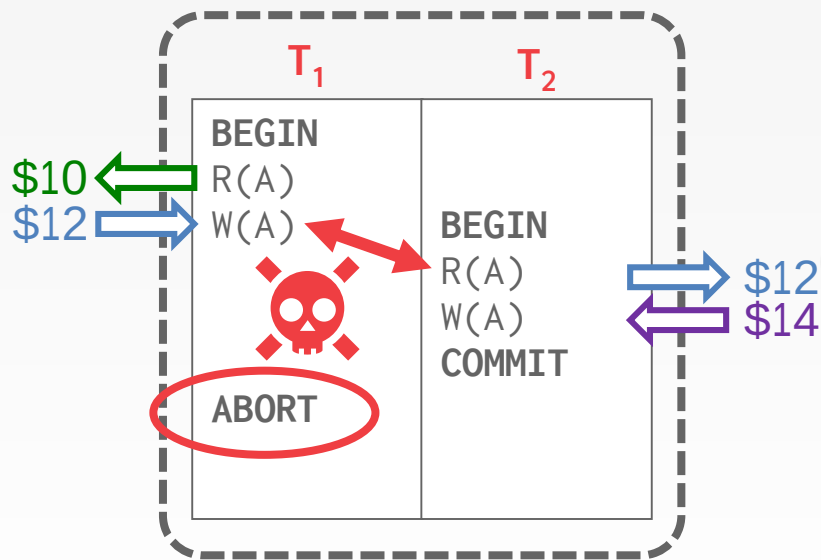
READ-WRITE CONFLICTS

Unrepeatable Reads



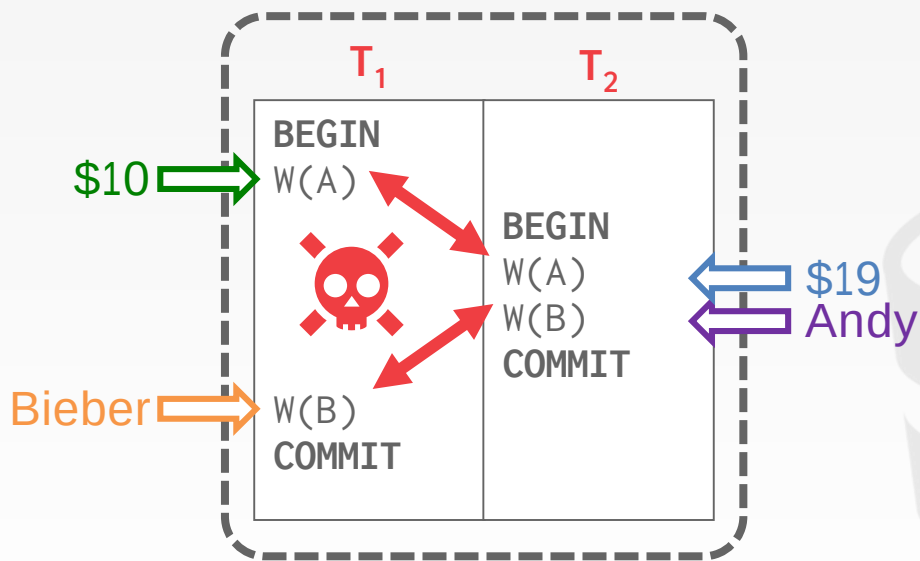
WRITE-READ CONFLICTS

Reading Uncommitted Data ("Dirty Reads")



WRITE-WRITE CONFLICTS

Overwriting Uncommitted Data



FORMAL PROPERTIES OF SCHEDULES

Given these conflicts, we now can understand what it means for a schedule to be serializable.

- This is to check whether schedules are correct.
- This is not how to generate a correct schedule.

There are different levels of serializability:

- **Conflict Serializability**
- **View Serializability**

Most DBMSs try to support this.

No DBMS can do this.

CONFLICT SERIALIZABLE SCHEDULES

Two schedules are **conflict equivalent** iff:

- They involve the same actions of the same transactions,
and
- Every pair of conflicting actions is ordered the same way.

Schedule **S** is **conflict serializable** if:

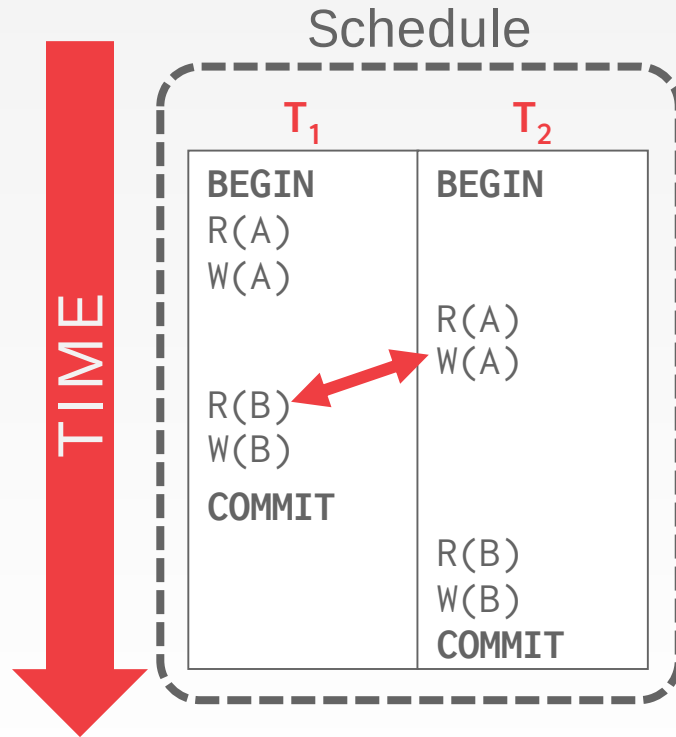
- **S** is conflict equivalent to some serial schedule.

CONFLICT SERIALIZABILITY INTUITION

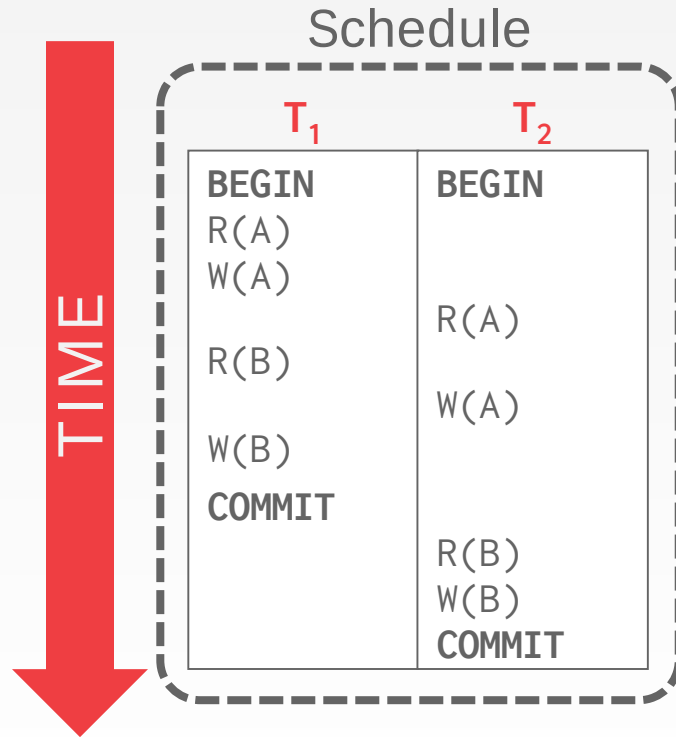
Schedule **S** is conflict serializable if you are able to transform **S** into a serial schedule by swapping consecutive non-conflicting operations of different transactions.



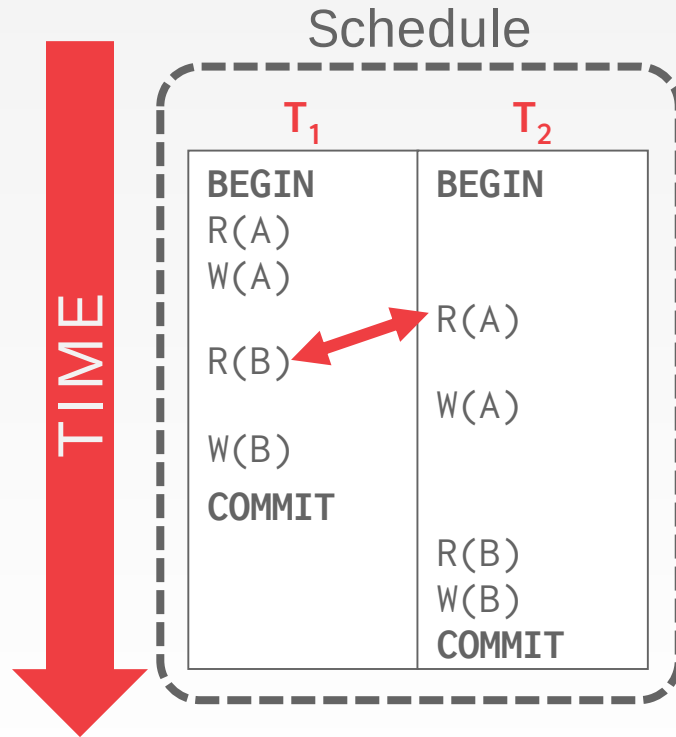
CONFLICT SERIALIZABILITY INTUITION



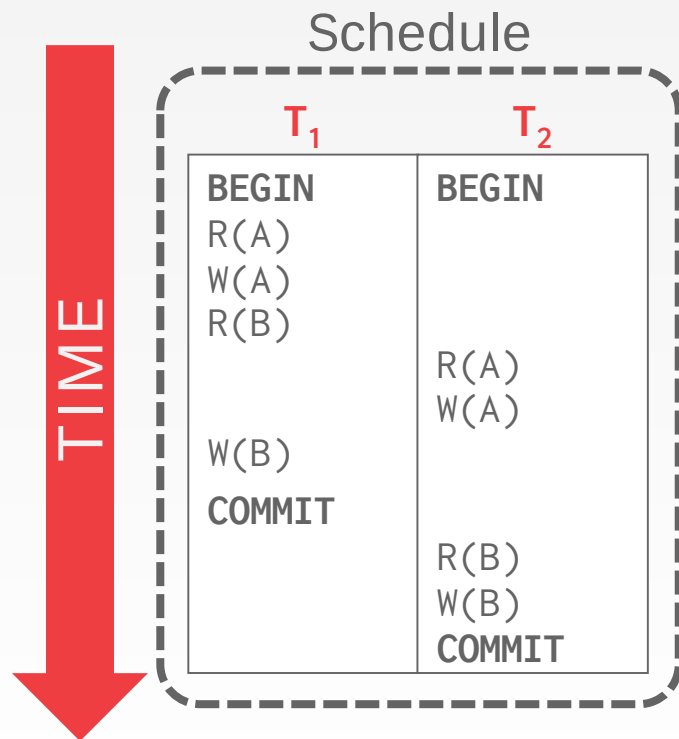
CONFLICT SERIALIZABILITY INTUITION



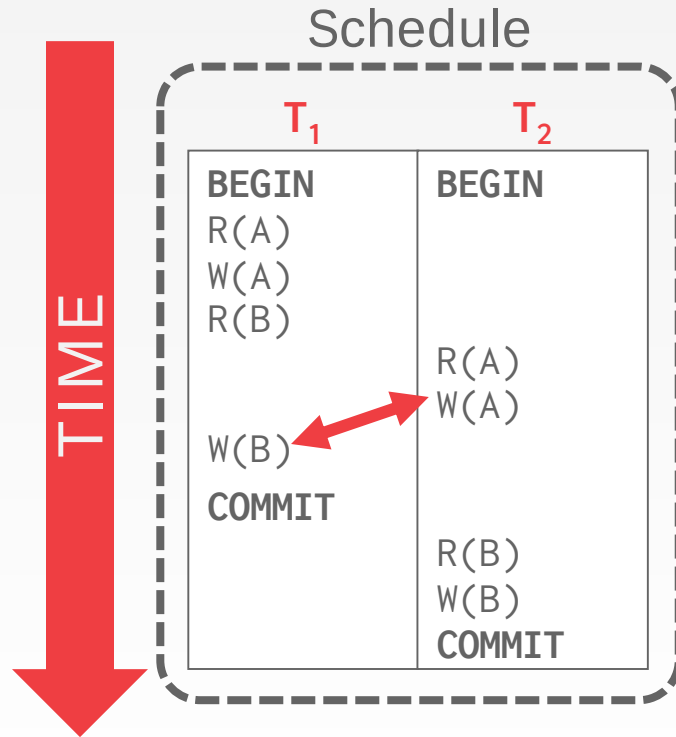
CONFLICT SERIALIZABILITY INTUITION



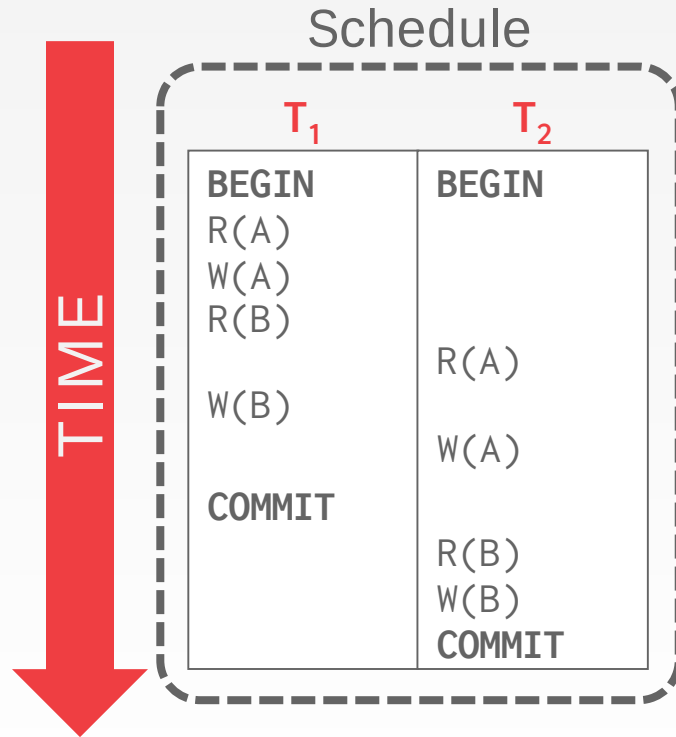
CONFLICT SERIALIZABILITY INTUITION



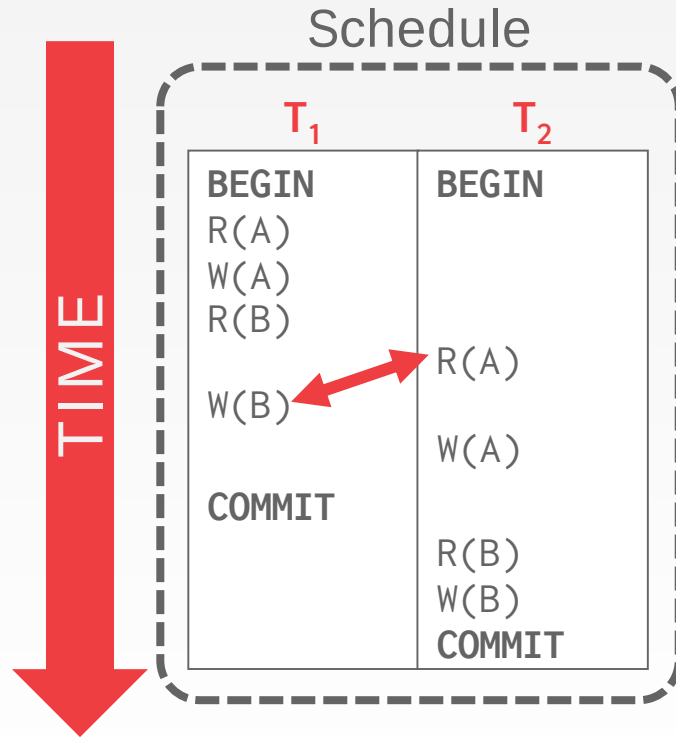
CONFLICT SERIALIZABILITY INTUITION



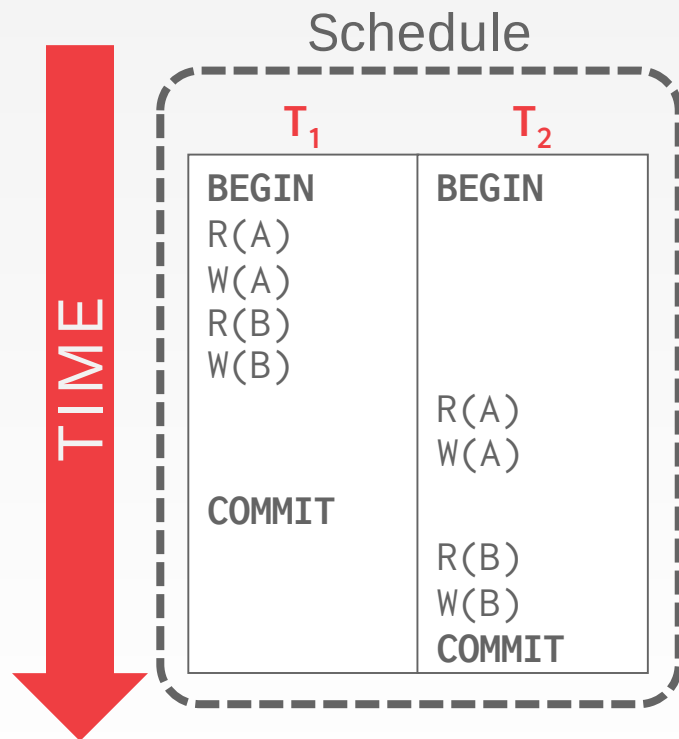
CONFLICT SERIALIZABILITY INTUITION



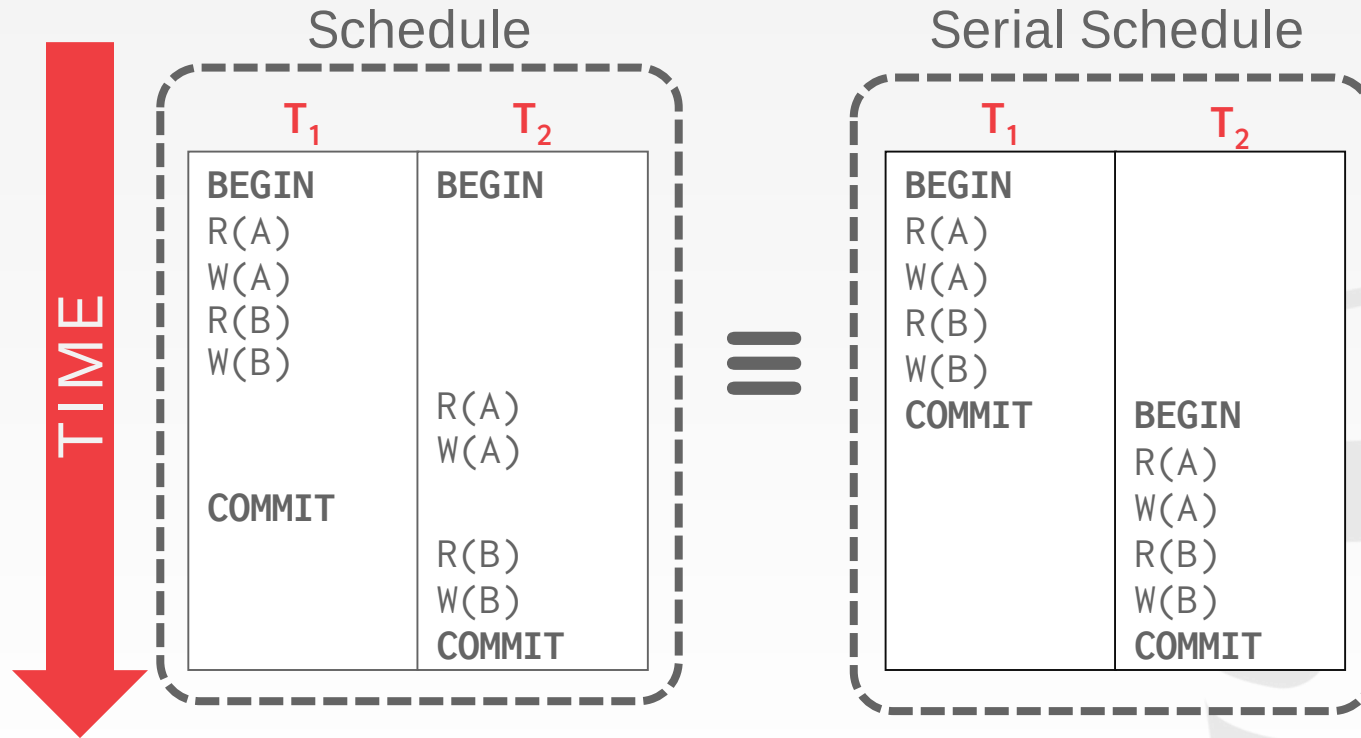
CONFLICT SERIALIZABILITY INTUITION



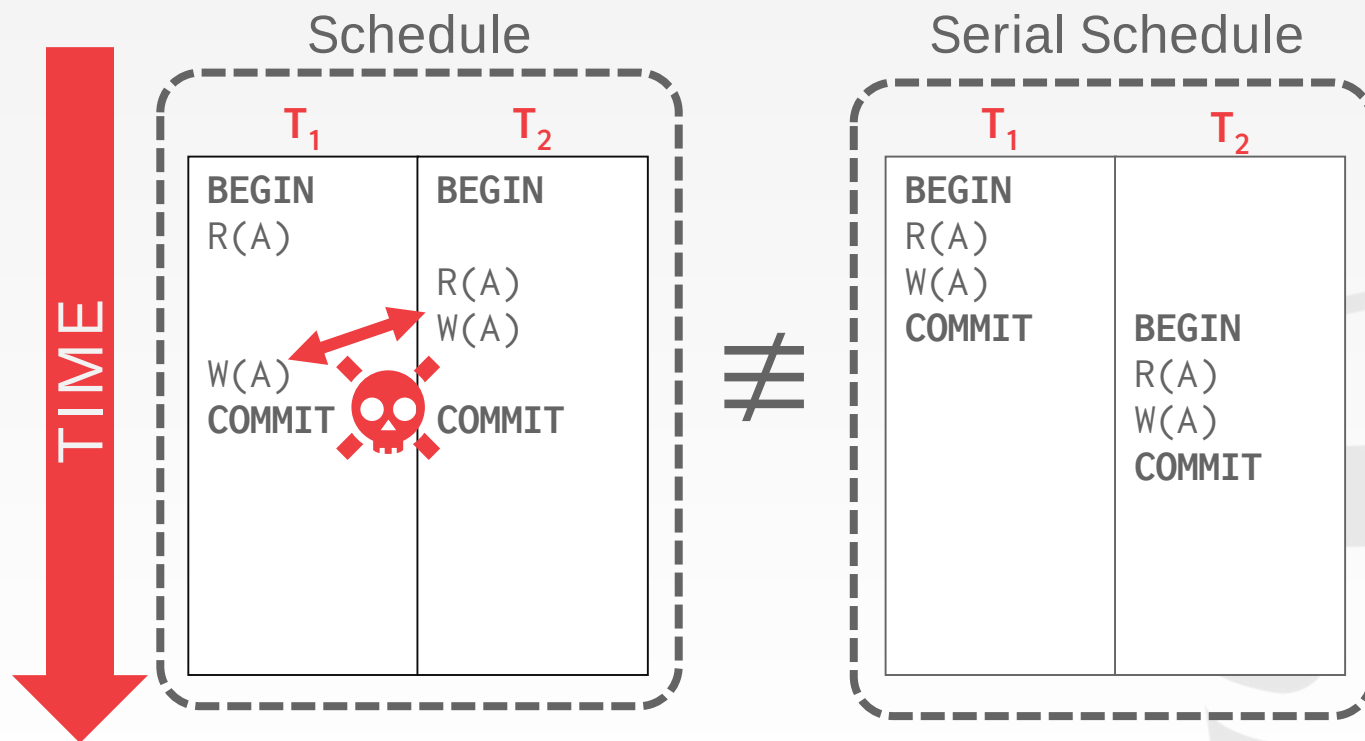
CONFLICT SERIALIZABILITY INTUITION



CONFLICT SERIALIZABILITY INTUITION



CONFLICT SERIALIZABILITY INTUITION



SERIALIZABILITY

Swapping operations is easy when there are only two txns in the schedule. It's cumbersome when there are many txns.

Are there any faster algorithms to figure this out other than transposing operations?

DEPENDENCY GRAPHS

One node per txn.

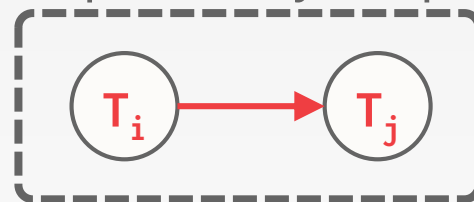
Edge from T_i to T_j if:

- An operation O_i of T_i conflicts with an operation O_j of T_j and
- O_i appears earlier in the schedule than O_j .

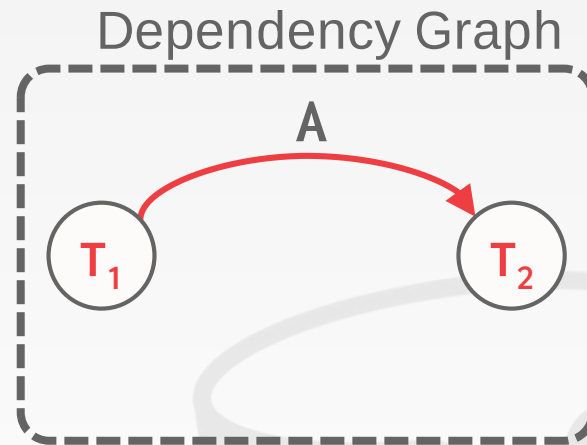
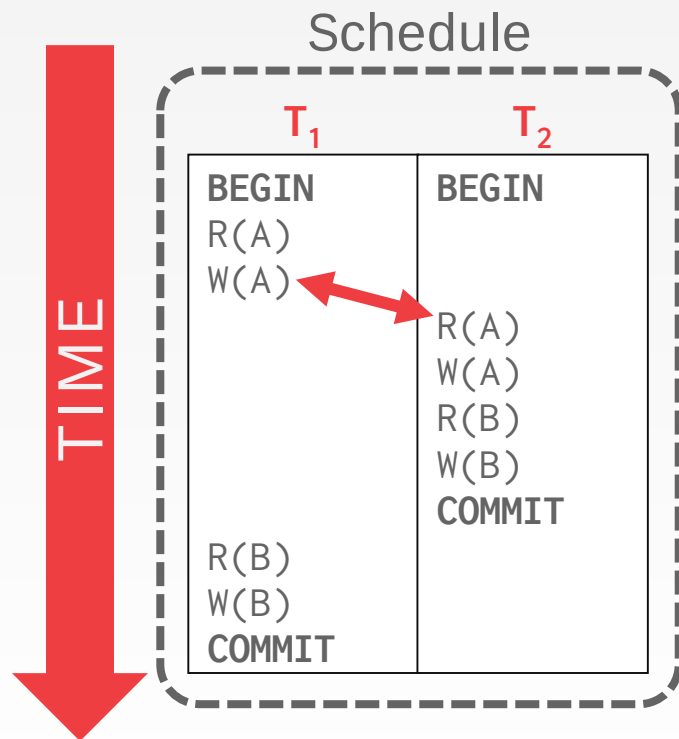
Also known as a **precedence graph**.

A schedule is conflict serializable iff its dependency graph is acyclic.

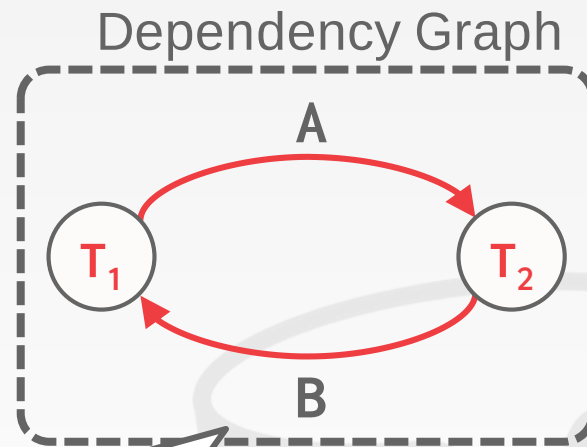
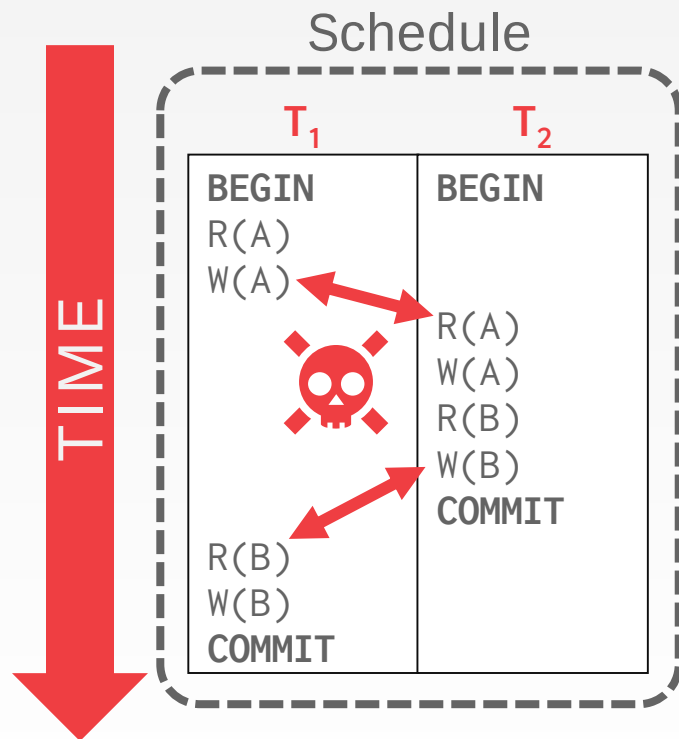
Dependency Graph



EXAMPLE #1

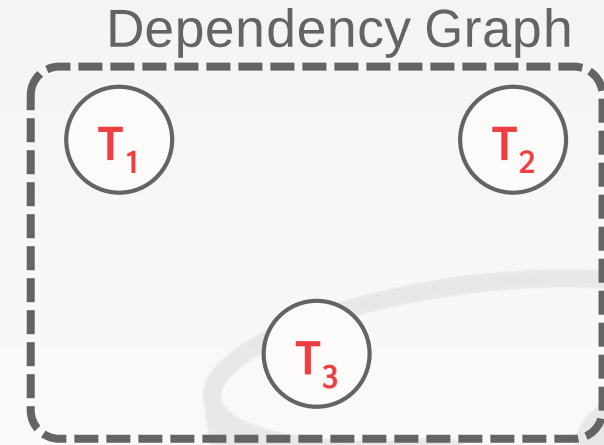
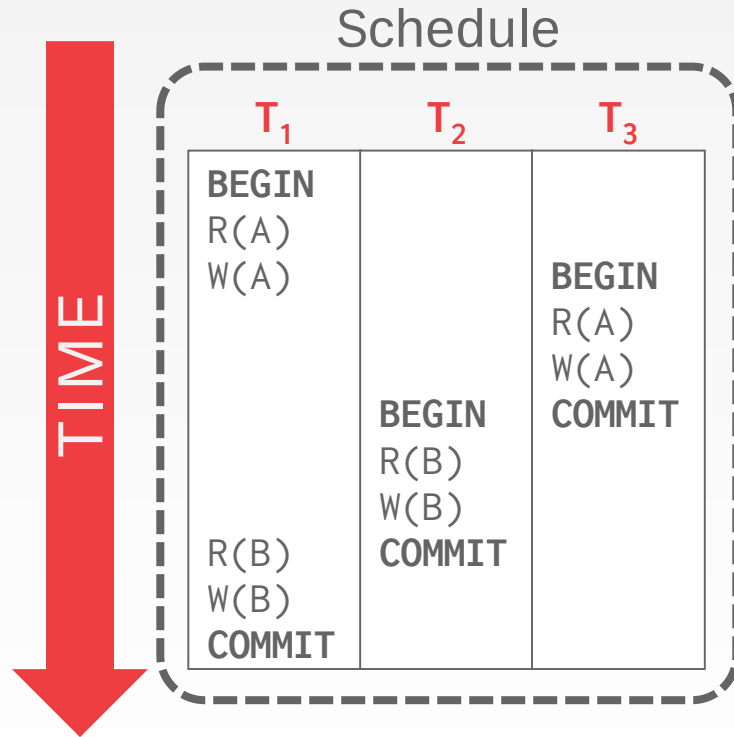


EXAMPLE #1

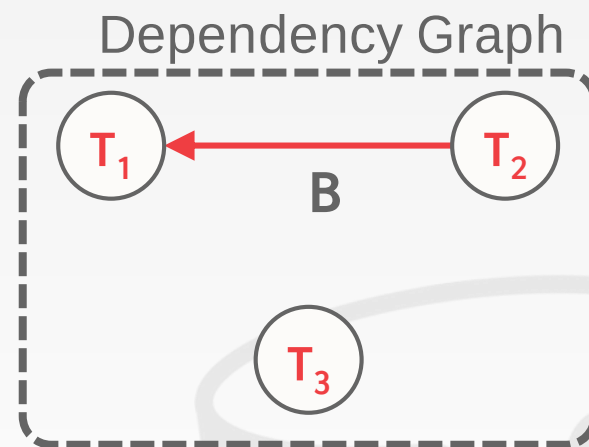
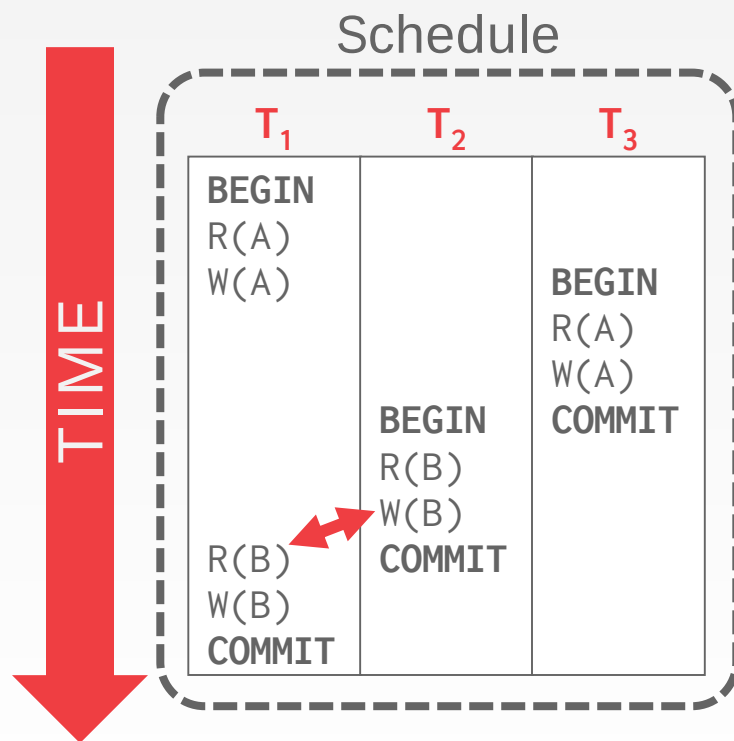


The cycle in the graph reveals the problem. The output of T_1 depends on T_2 , and vice-versa.

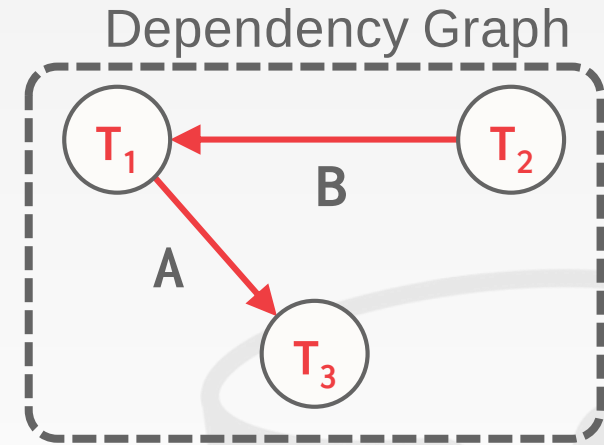
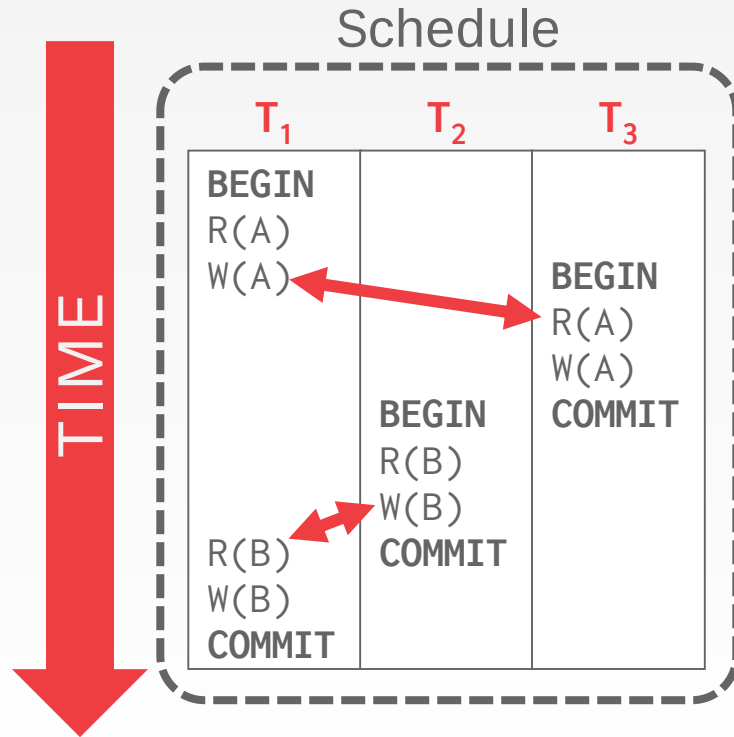
EXAMPLE #2 – THREESOME



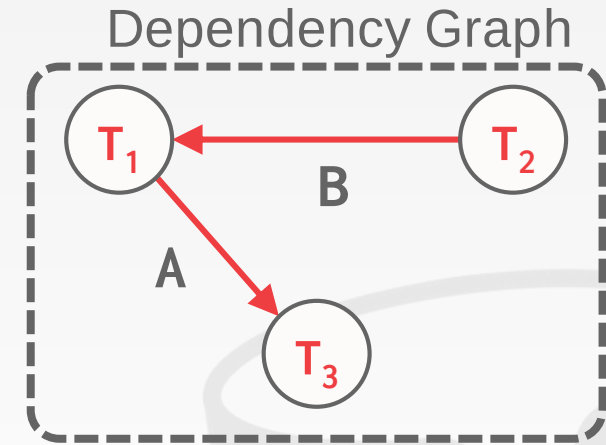
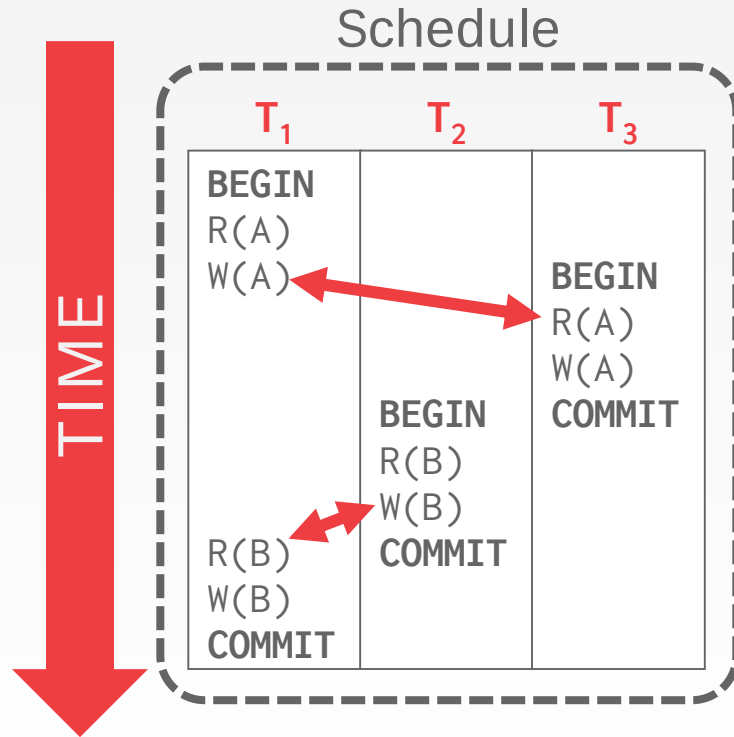
EXAMPLE #2 – THREESOME



EXAMPLE #2 – THREESOME

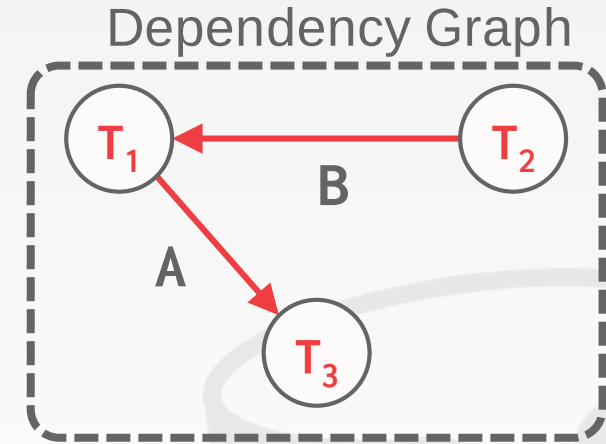
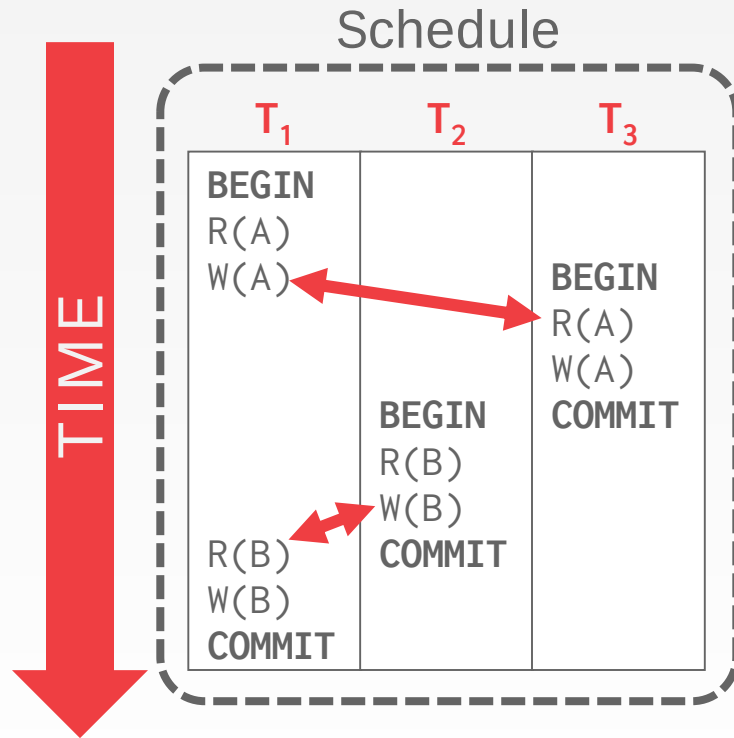


EXAMPLE #2 – THREESOME



Is this equivalent to a serial execution?

EXAMPLE #2 – THREESOME

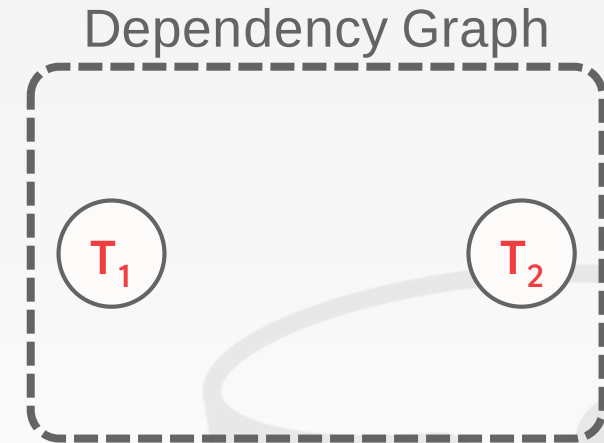
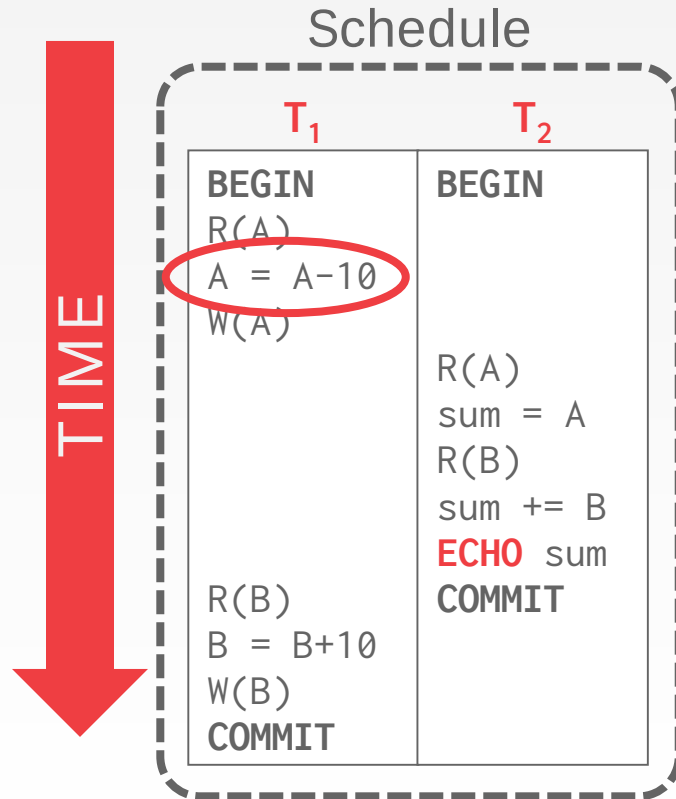


Is this equivalent to a serial execution?

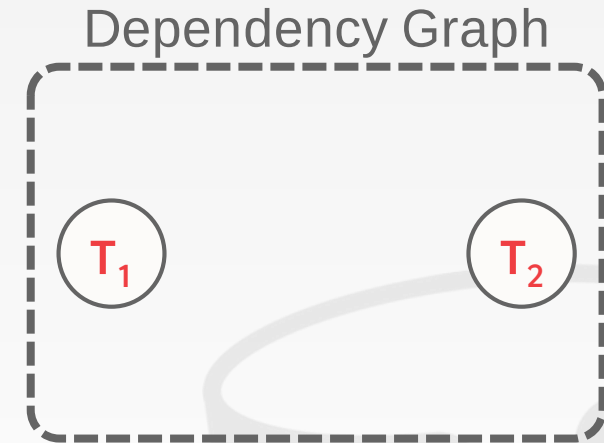
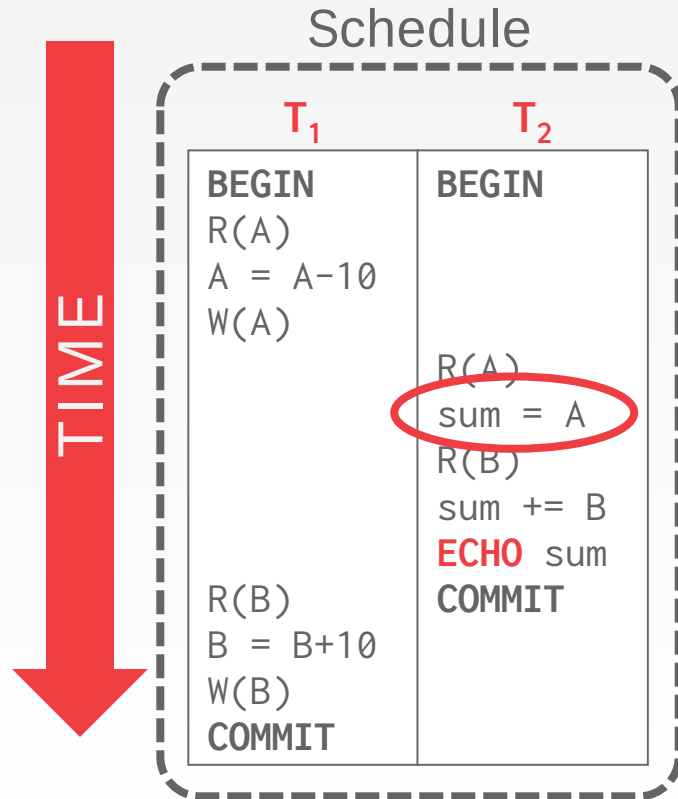
Yes (T_2, T_1, T_3)

→ Notice that T_3 should go after T_2 , although it starts before it!

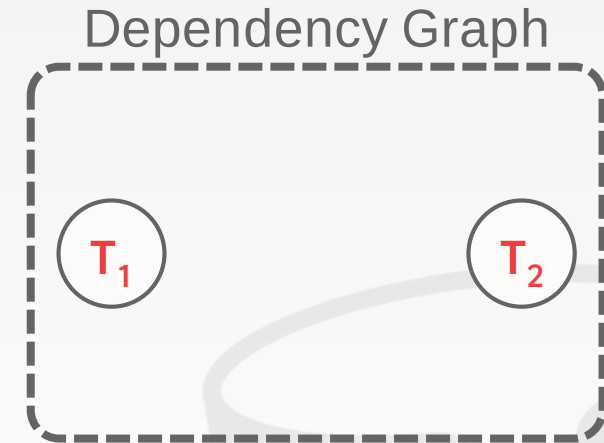
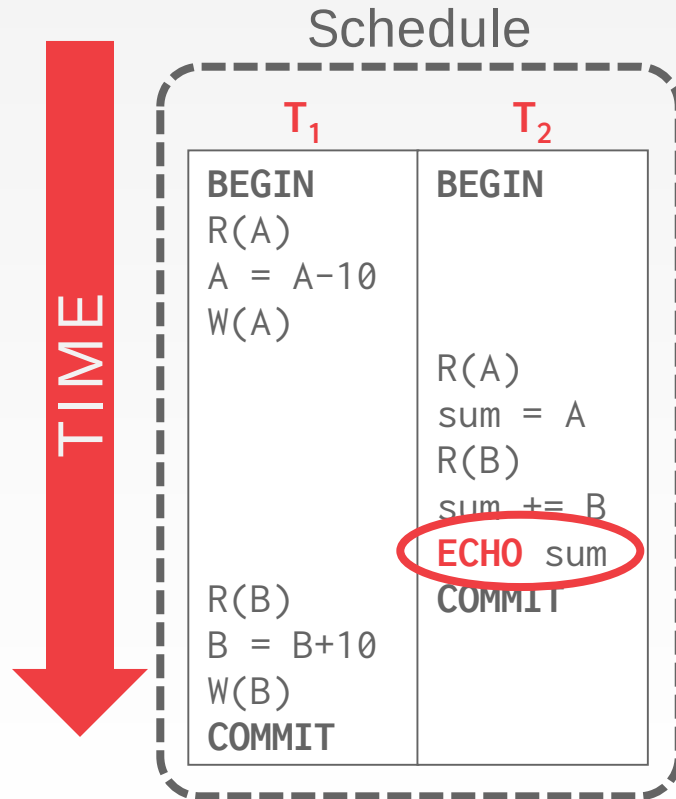
EXAMPLE #3 – INCONSISTENT ANALYSIS



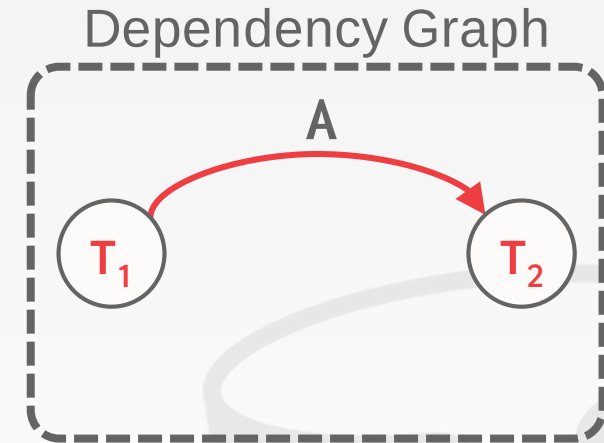
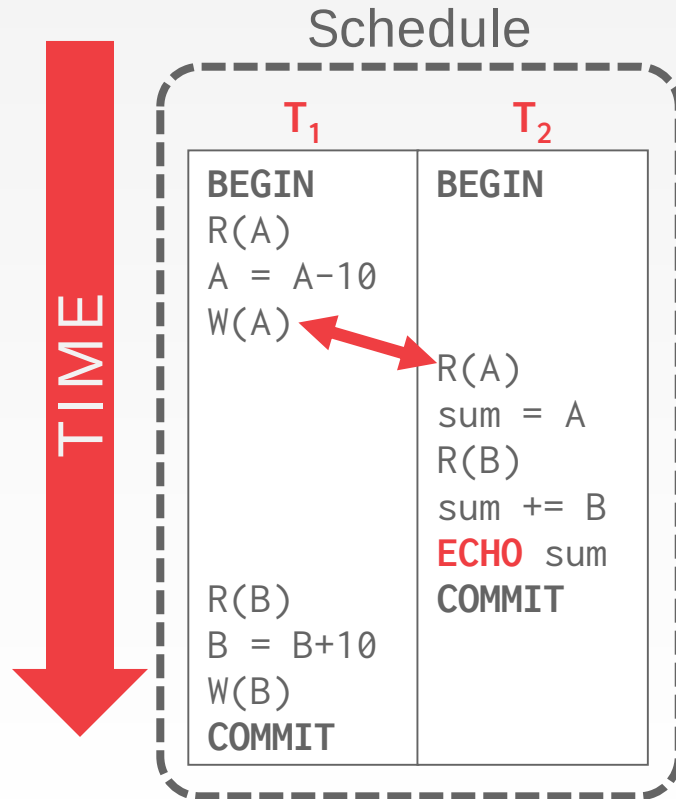
EXAMPLE #3 – INCONSISTENT ANALYSIS



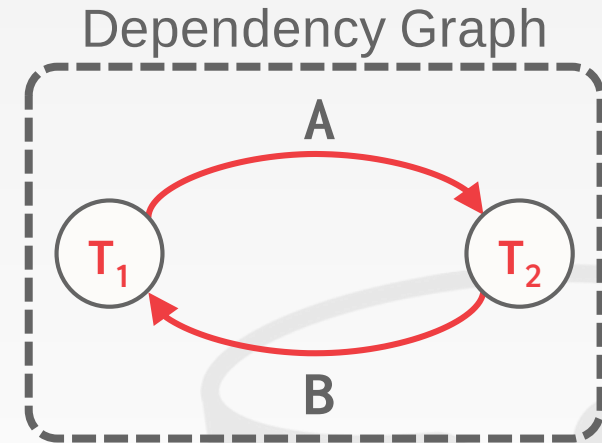
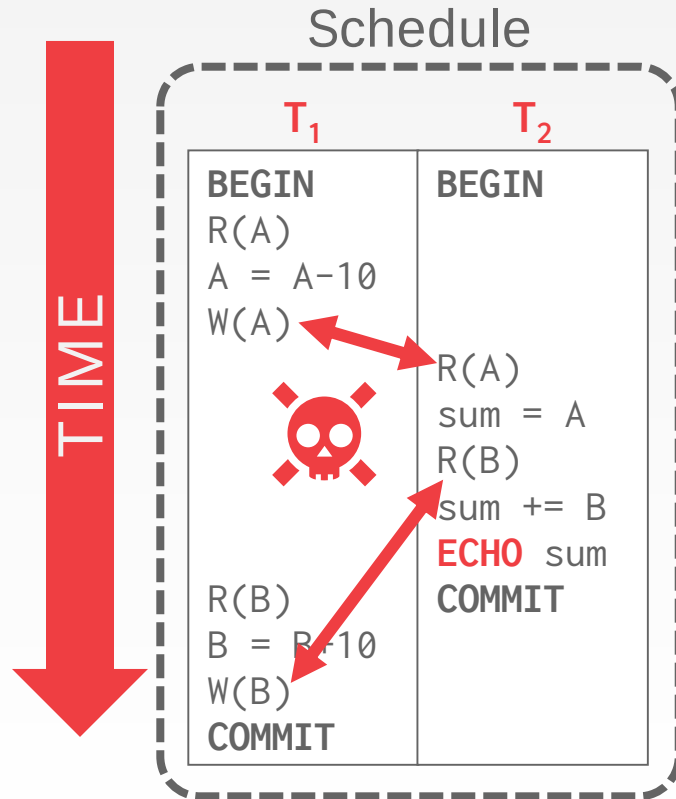
EXAMPLE #3 – INCONSISTENT ANALYSIS



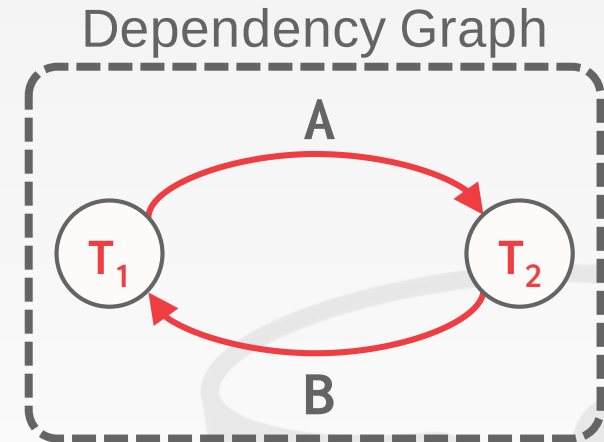
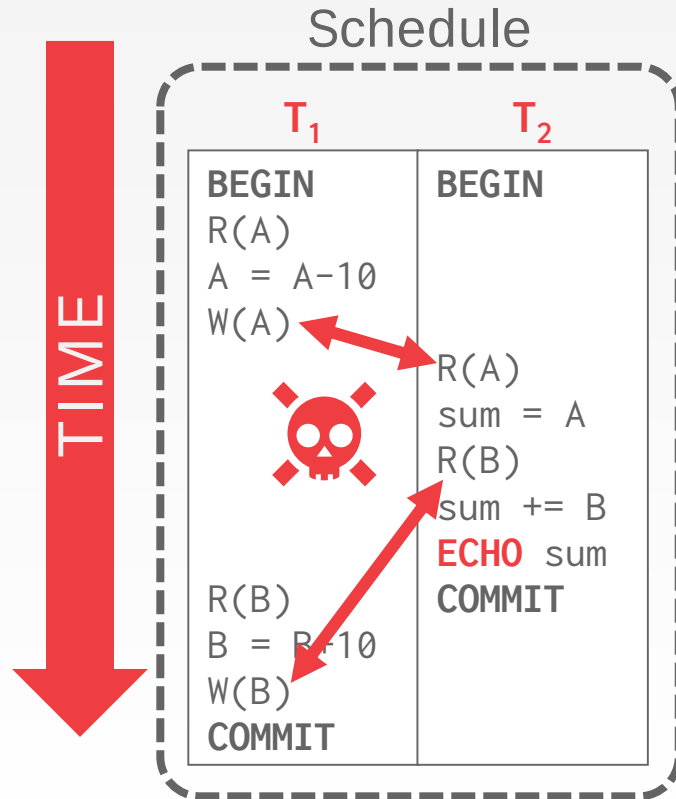
EXAMPLE #3 – INCONSISTENT ANALYSIS



EXAMPLE #3 – INCONSISTENT ANALYSIS

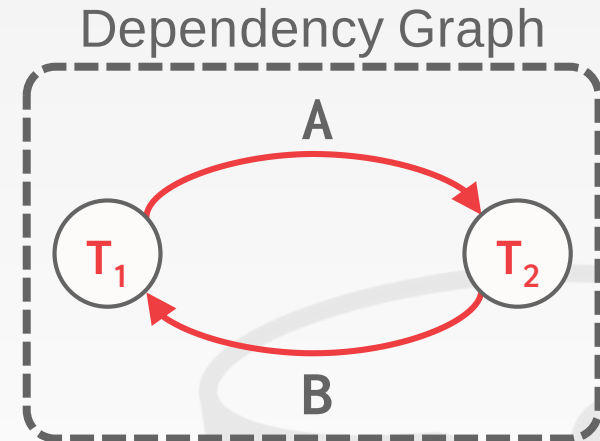
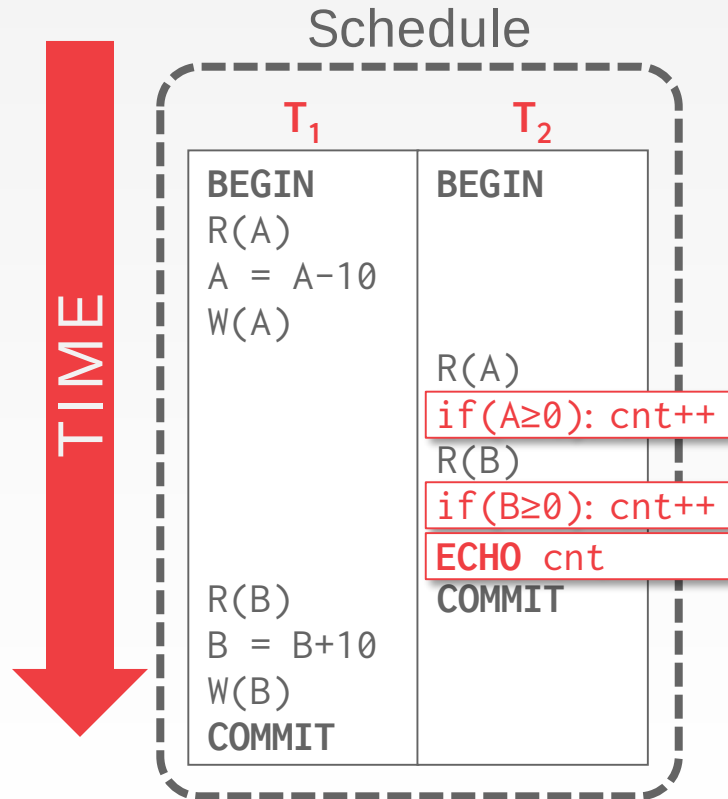


EXAMPLE #3 – INCONSISTENT ANALYSIS



Is it possible to modify only the application logic so that schedule produces a "correct" result but is still not conflict serializable?

EXAMPLE #3 – INCONSISTENT ANALYSIS



Is it possible to modify only the application logic so that schedule produces a "correct" result but is still not conflict serializable?

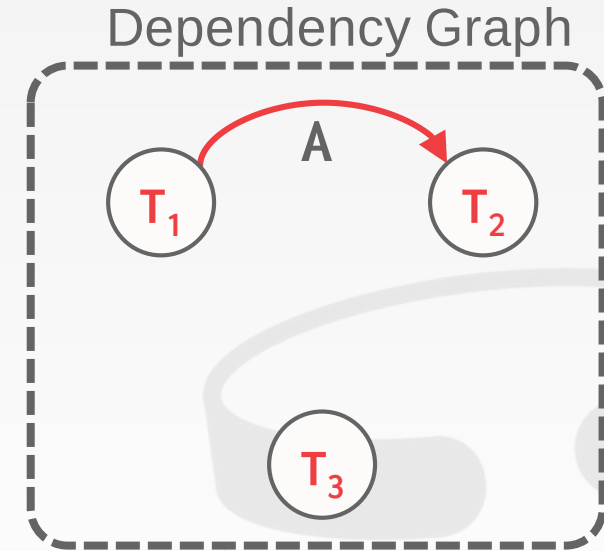
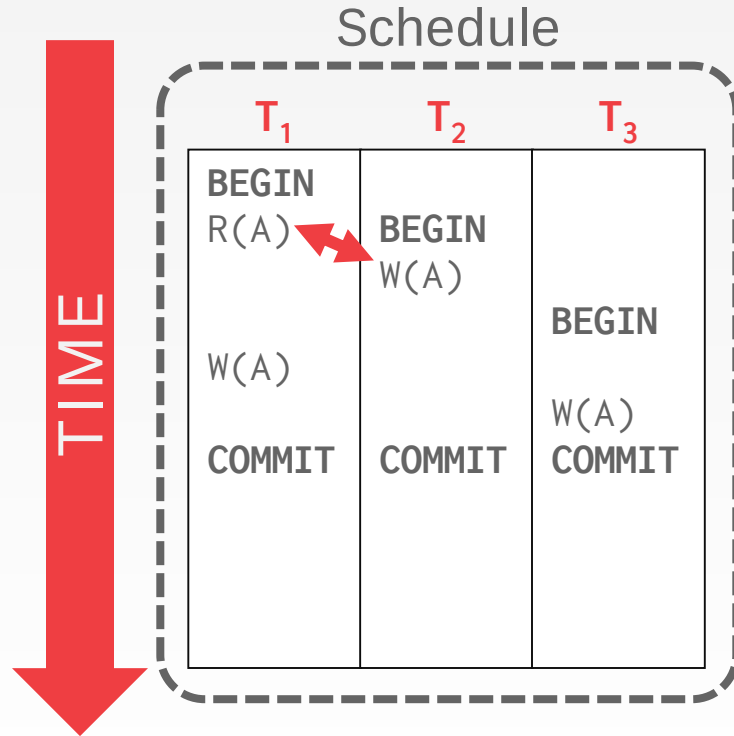
VIEW SERIALIZABILITY

Alternative (weaker) notion of serializability.

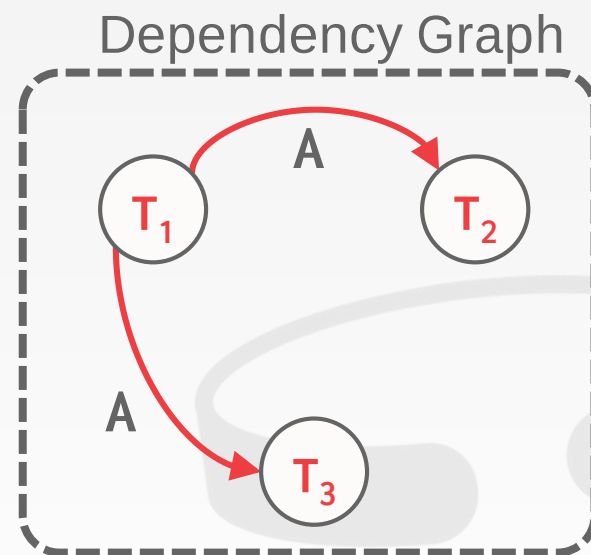
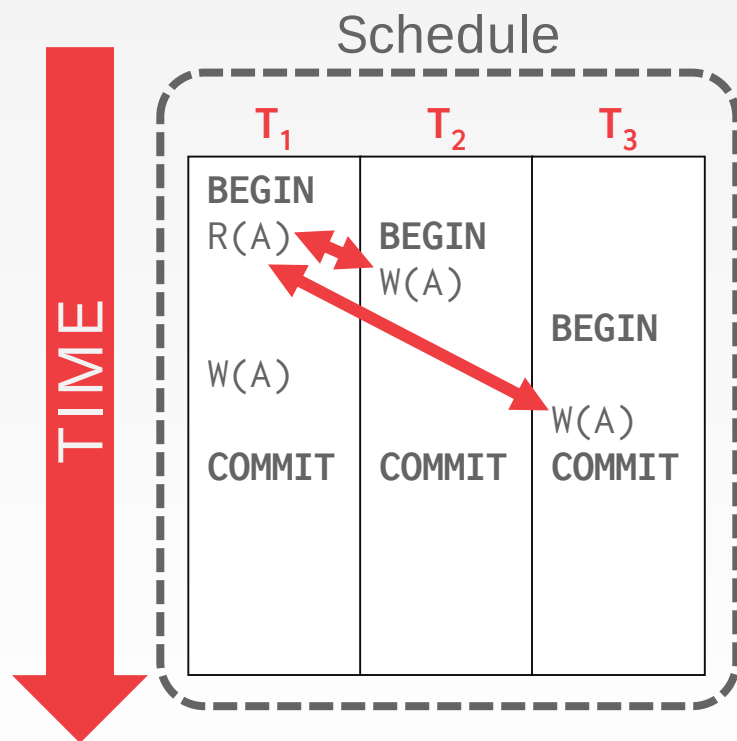
Schedules S_1 and S_2 are view equivalent if:

- If T_1 reads initial value of A in S_1 , then T_1 also reads initial value of A in S_2 .
- If T_1 reads value of A written by T_2 in S_1 , then T_1 also reads value of A written by T_2 in S_2 .
- If T_1 writes final value of A in S_1 , then T_1 also writes final value of A in S_2 .

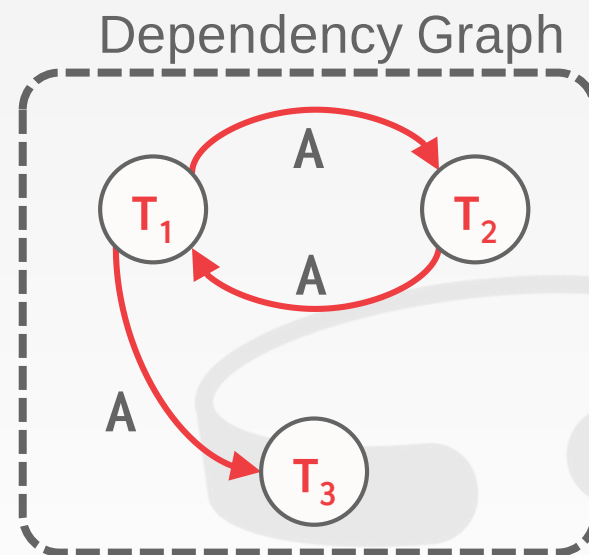
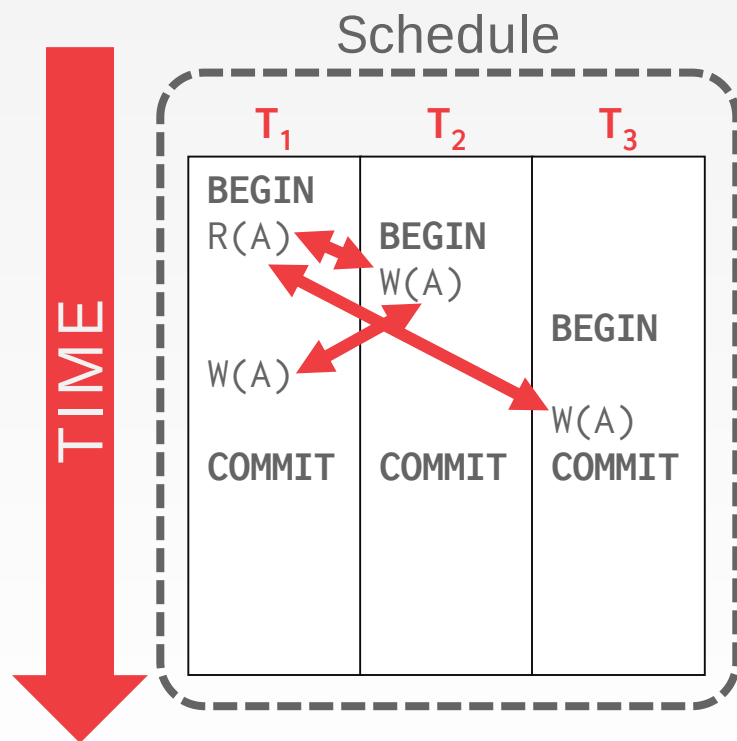
VIEW SERIALIZABILITY



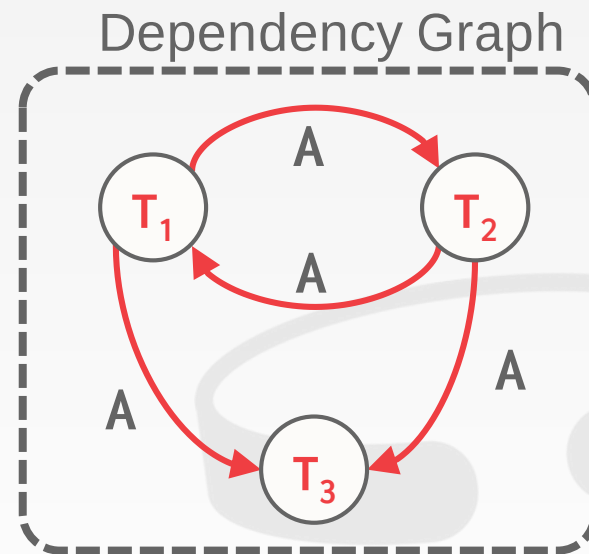
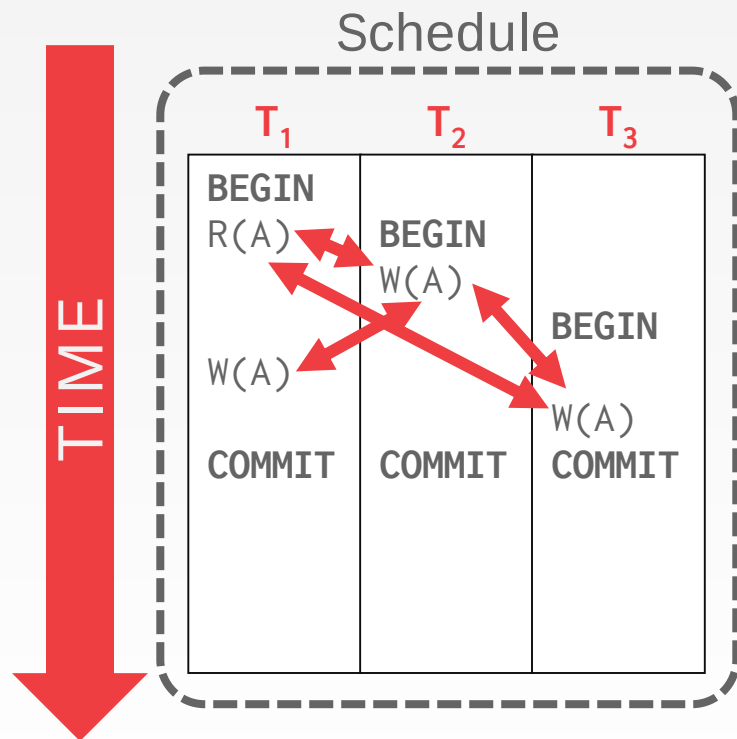
VIEW SERIALIZABILITY



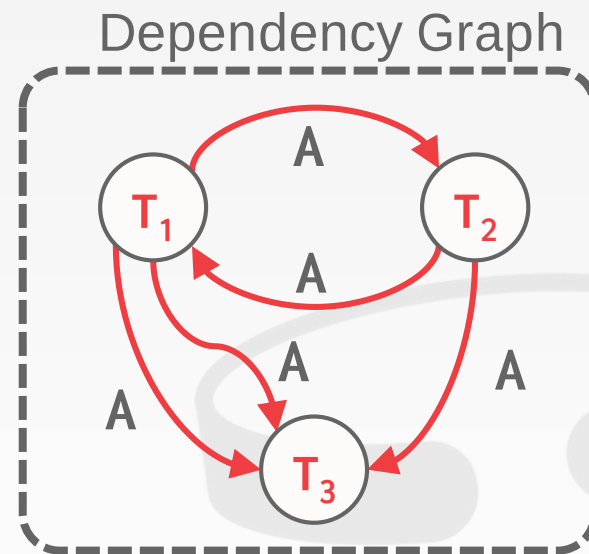
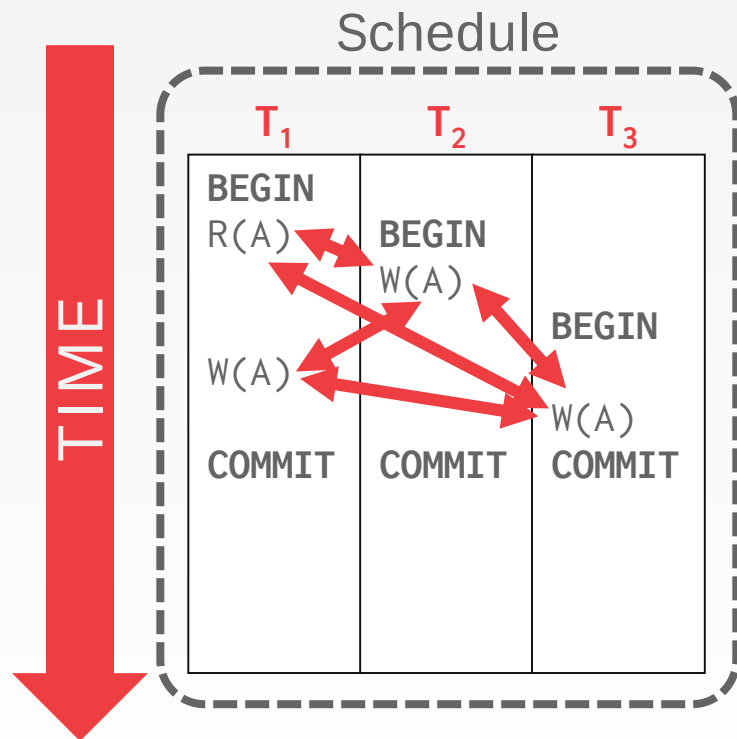
VIEW SERIALIZABILITY



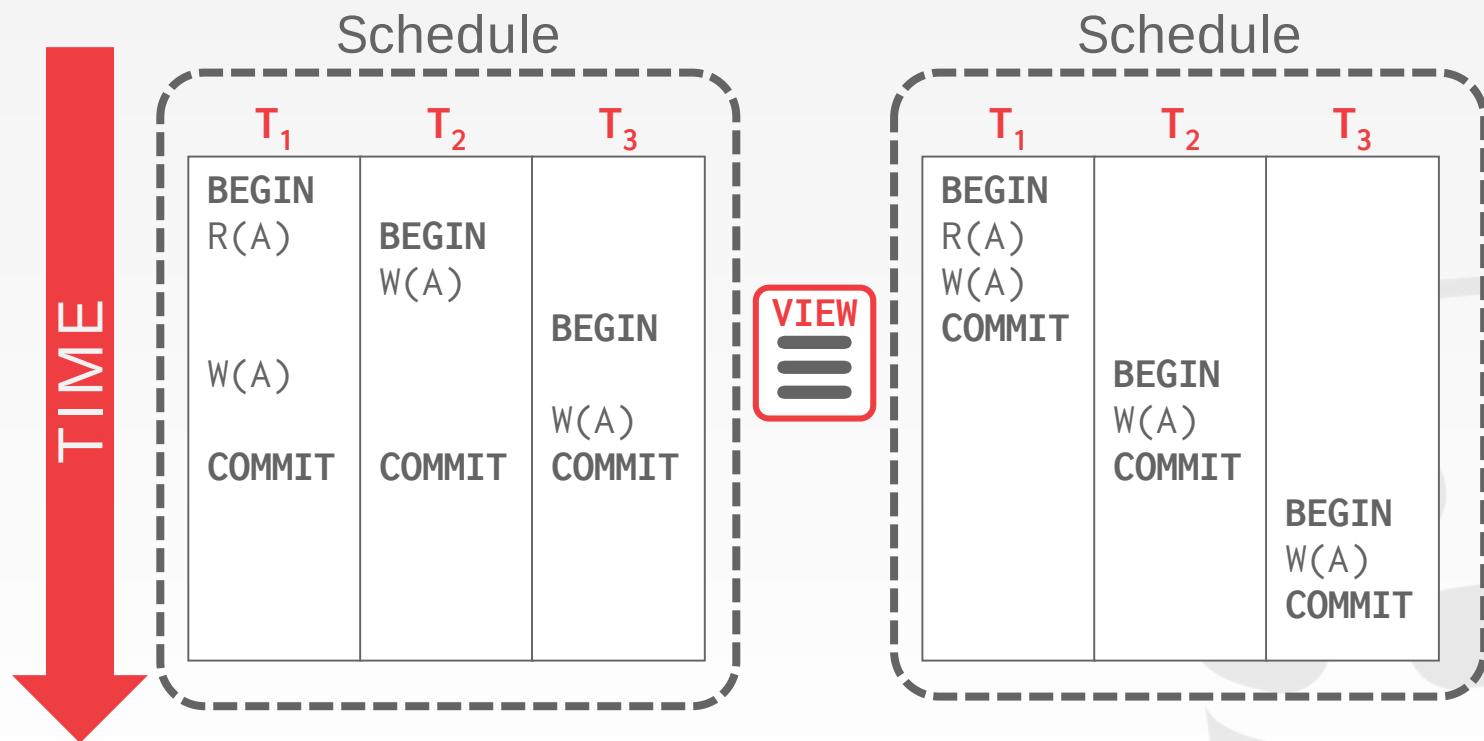
VIEW SERIALIZABILITY



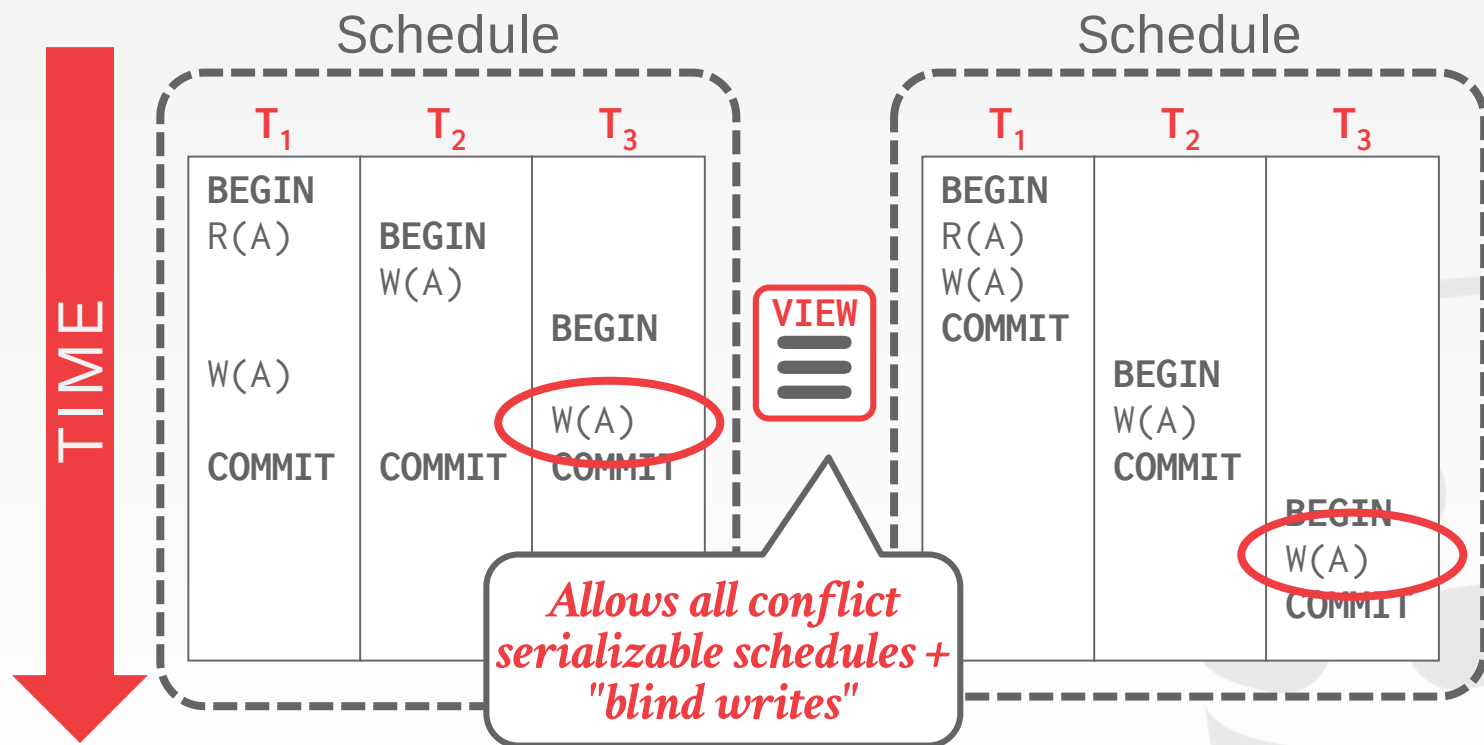
VIEW SERIALIZABILITY



VIEW SERIALIZABILITY



VIEW SERIALIZABILITY



SERIALIZABILITY

View Serializability allows for (slightly) more schedules than **Conflict Serializability** does.

→ But is difficult to enforce efficiently.

Neither definition allows all schedules that you would consider "serializable".

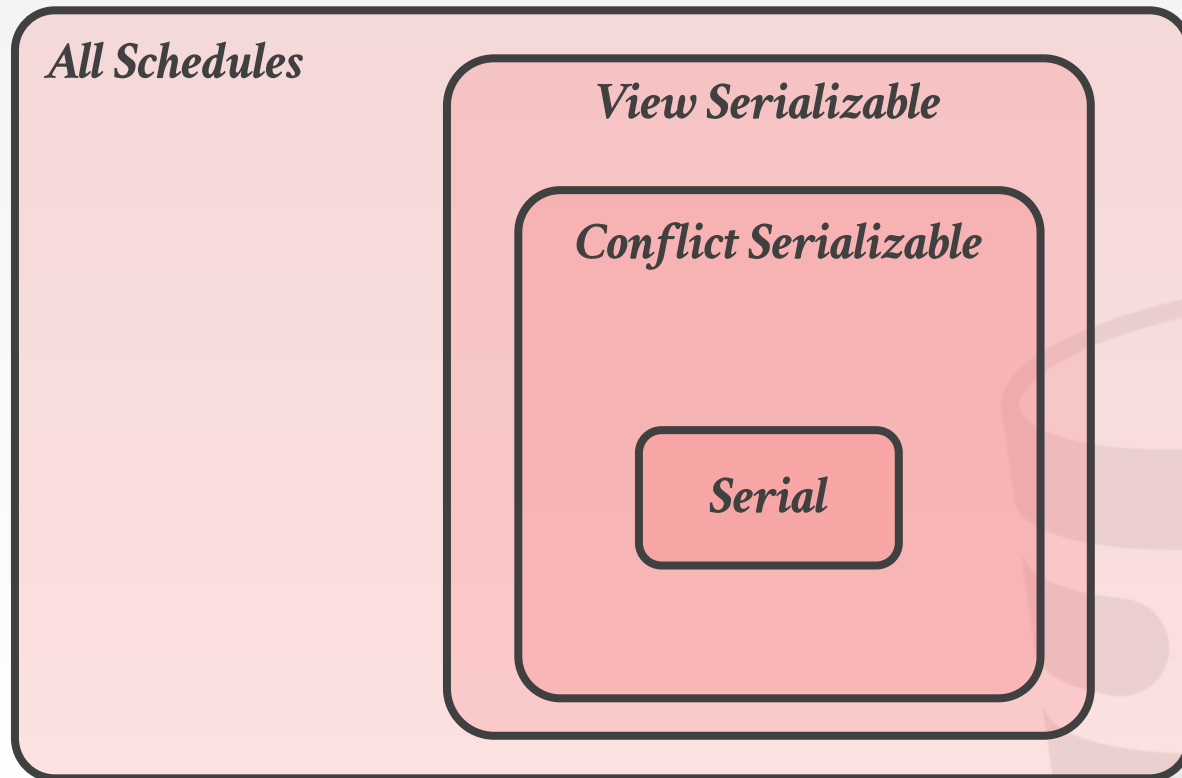
→ This is because they don't understand the meanings of the operations or the data (recall example #3)

SERIALIZABILITY

In practice, **Conflict Serializability** is what systems support because it can be enforced efficiently.

To allow more concurrency, some special cases get handled separately at the application level.

UNIVERSE OF SCHEDULES



TRANSACTION DURABILITY

All of the changes of committed transactions should be persistent.

- No torn updates.
- No changes from failed transactions.

The DBMS can use either logging or shadow paging to ensure that all changes are durable.

ACID PROPERTIES

Atomicity: All actions in the txn happen, or none happen.

Consistency: If each txn is consistent and the DB starts consistent, then it ends up consistent.

Isolation: Execution of one txn is isolated from that of other txns.

Durability: If a txn commits, its effects persist.

CONCLUSION

Concurrency control and recovery are among the most important functions provided by a DBMS.

Concurrency control is automatic

- System automatically inserts lock/unlock requests and schedules actions of different txns.
- Ensures that resulting execution is equivalent to executing the txns one after the other in some order.

CONCLUS

Concurrency control and recovery are the most important functions provided by a database. Concurrency control is automated. → System automatically inserts locks and schedules actions of different transactions.

ability problems that it brings [9, 10, 19]. We believe it is better to have application programmers deal with performance problems due to overuse of transactions as bottlenecks arise, rather than always coding around the lack of transactions. Running two-phase commit over Paxos

Spanner: Google's Globally-Distributed Database

James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymbanski, Christopher Taylor, Ruth Wang, Dale Woodford

Google, Inc.

Abstract

Spanner is Google's scalable, multi-version, globally-distributed, and synchronously-replicated database. It is the first system to distribute data at global scale and support externally-consistent distributed transactions. This paper describes how Spanner is structured, its feature set, the rationale underlying various design decisions, and a novel time API that exposes clock uncertainty. This API and its implementation are critical to supporting external consistency and a variety of powerful features: non-blocking reads in the past, lock-free read-only transactions, and atomic schema changes, across all of Spanner.

tenacy over higher availability, as long as they can survive 1 or 2 datacenter failures.

Spanner's main focus is managing cross-datacenter replicated data, but we have also spent a great deal of time in designing and implementing important database features on top of our distributed-systems infrastructure. Even though many projects happily use Bigtable [9], we have also consistently received complaints from users that Bigtable can be difficult to use for some kinds of applications: those that have complex, evolving schemas, or those that want strong consistency in the presence of wide-area replication. (Similar claims have been made by other authors [37].) Many applications at Google have chosen to use Megastore [5] because of its semi-relational data model and support for synchronous replication, despite its relatively poor write throughput. As a result, Spanner has evolved from a Bigtable-like key-value store into a temporal multi-version database. Data is stored in a schematized semi-relational model, is versioned, and each version is automatically stamped with its commit time; old versions of data are subject to configurable garbage-collection policies. Applications can read data at old timestamps, supports general-purpose transactions, and provides a query language.

Spanner is a globally-distributed database. Spanner provides a rich set of features. First, the replication control algorithm can be dynamically controlled at the application level. Applications can specify control policies for which datacenters contain which data, and which users (to control read latency), and which datacenters are from each other (to control write latency). Second, many replicas are maintained (to control availability, and read performance). Data is managed dynamically and transparently moved between datacenters by the system to balance resource usage. Third, Spanner has two features that are critical to implementing a distributed database: it

1 Introduction

CONCLUSION

Concurrency control and recovery are among the most important functions provided by a DBMS.

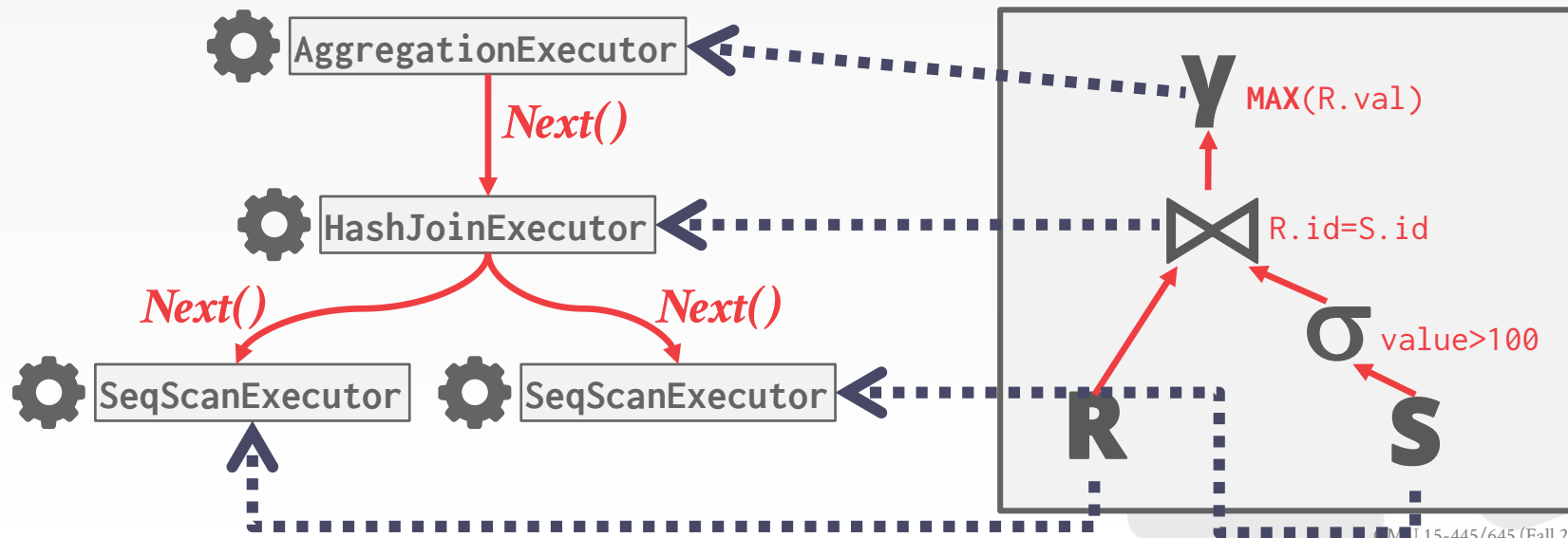
Concurrency control is automatic

- System automatically inserts lock/unlock requests and schedules actions of different txns.
- Ensures that resulting execution is equivalent to executing the txns one after the other in some order.

PROJECT #3

You will build a query execution engine in your DBMS.

```
SELECT MAX(R.val)
FROM R JOIN S
ON R.id = S.id
WHERE S.value > 100
```



PROJECT #3 – TASKS

Install Tables in Catalog

Plan Node Executors

- Insert
- Sequential Scan
- Hash Join
- Hash Aggregation

<https://15445.courses.cs.cmu.edu/fall2019/project2/>

DEVELOPMENT HINTS

You do not need a working Linear Probe Hash Table to complete Tasks #1 and #2.

Implement the insert executor first.

You do not need to worry about transactions.

Gradescope is for meant for grading, not debugging. Write your own local tests.

THINGS TO NOTE

Do **not** change any file other than the ones that you submit to Gradescope.

Rebase on top of the latest BusTub master branch.

Post your questions on Piazza or come to TA office hours.

PLAGIARISM WARNING

Your project implementation must be your own work.

- You may not copy source code from other groups or the web.
- Do not publish your implementation on Github.

Plagiarism will not be tolerated.
See [CMU's Policy on Academic Integrity](#) for additional information.



NEXT CLASS

Two-Phase Locking
Isolation Levels



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]